

Министерство образования Новосибирской области
ГБПОУ НСО «Новосибирский авиационный технический колледж
имени Б.С. Галуцака»

Разработка веб-приложения для декомпозиции целей

Пояснительная записка к дипломному проекту

НАТКиГ.101700.010.000ПЗ

Разработал:
студент группы ПР-23.106
Мешков А.А.

Содержание

Введение.....	3
1 Исследование предметной области	5
1.1 Описание предметной области	5
1.2 Определение функциональных задач пользователей.....	8
1.3 Анализ аналогов и прототипов	11
1.4 Проектирование информационной системы	17
1.5 Анализ программных ресурсов, необходимых в работе.....	21
2 Технологический раздел.....	24
2.1 Структура базы данных	24
2.2 Разработка интерфейса программного продукта.....	25
2.3 Разработка структуры программного продукта.....	31
2.4 Описание процедур и функций.....	32
3 Рекомендации по эксплуатации продукта	37
3.1 Назначение и условия применения	37
3.2 Подготовка к работе.....	38
4 Тестирование программного продукта	44
4.1 Выбор стратегии тестирования.....	44
4.2 Разработка методов тестирования.....	45
4.3 Протоколы тестирования	56
Заключение	58
Библиография	59
Приложение А (обязательное) ER-диаграмма	60
Приложение Б (обязательное) Словарь данных.....	61

					НАТКиГ.101700.010.000ПЗ			
Изм.	Лист	№ докум	Подпись	Дата				
Разраб		Мешиков А.А.			Разработка веб-приложения для декомпозиции целей	Литера	Лист	Листов
Пров		Иванов А.С..				у	2	63
Н. Контр		Гербер М.Р.				ПР-23.106		
Утв		Тышкевич Е.В.						

Введение

Современное общество характеризуется быстрым ростом объёма информации, усложнением профессиональной деятельности и увеличением количества задач, с которыми сталкивается человек в учебной и повседневной жизни. В таких условиях особую значимость приобретают инструменты, позволяющие структурировать деятельность и эффективно организовывать процесс достижения поставленных целей, включая командную работу.

Постановка крупных и долгосрочных целей часто сопровождается неопределённостью, отсутствием чёткого плана действий и сложностью разбиения общей задачи на последовательные этапы. Подобные трудности могут приводить к снижению мотивации, прокрастинации и затруднениям при планировании личной или профессиональной деятельности. Указанные проблемы усугубляются, когда речь идёт о коллективной работе: синхронизация усилий нескольких участников, распределение ответственности и поддержание единого видения целей требуют дополнительных организационных усилий. Кроме того, различия в уровне подготовки и подходах участников команды могут затруднять согласование действий и принятие решений.

Развитие цифровых технологий и веб-приложений открывает новые возможности для создания инструментов, способных помогать пользователям в формулировании целей, их структурировании и поэтапной реализации как в индивидуальном, так и в командном режимах.

Основная идея приложения заключается в создании инструмента, позволяющего вводить цель в свободной текстовой форме и получать структурированное представление возможных этапов её достижения с возможностью последующего ручного редактирования. Для реализации коллективного взаимодействия предусмотрены механизмы создания команд, общих списков задач и обмена планами между участниками, а также встроенные чаты для коммуникации в контексте текущих проектов. Функциональность

приложения охватывает регистрацию и аутентификацию пользователей, интерфейс ввода и декомпозиции целей, отслеживание прогресса и визуализацию структуры задач. Для генерации рекомендаций по декомпозиции предполагается использование внешних языковых сервисов через специализированный программный интерфейс. При этом система позиционируется как вспомогательный инструмент, формирующий ориентировочный план, который пользователь может адаптировать под индивидуальные потребности.

Целью разработки является создание веб-приложения, предназначенного для поддержки процесса постановки и структурирования целей, организации работы с задачами и осуществление командного пространства для работы и взаимодействия внутри команды. Для достижения поставленной цели предполагается выполнить ряд задач. К числу таких задач относится:

- изучить предметную область;
- определить функциональные задачи пользователей;
- провести анализ аналогов и прототипов с целью выявления положительных сторон;
- выполнить проектирование информационной системы;
- провести анализ программных ресурсов, необходимых для реализации;
- спроектировать структуру базы данных;
- разработать интерфейс программного продукта, структуру программного продукта, алгоритмы обработки информации, инструкции пользователям системы;
- выбрать стратегию тестирования;
- разработать сценарии тестирования;
- составить протоколы тестирования.

Таким образом, решение перечисленных задач позволит в полном объёме достичь поставленной цели и разработать веб-приложение для декомпозиции целей и организации работы с задачами, включая командное взаимодействие.

1 Исследование предметной области

1.1 Описание предметной области

Современная цифровая среда характеризуется большим количеством информации и постоянным ростом числа задач, с которыми сталкивается человек в учебной, профессиональной и личной деятельности. Планирование работы и достижение поставленных целей становятся важной частью повседневной жизни. Многие задачи требуют выполнения большого количества действий и могут занимать значительный промежуток времени, причём всё больше таких задач реализуется в коллективном формате, где успех зависит от согласованности действий участников.

Сложность планирования возрастает пропорционально количеству вовлечённых лиц, поскольку каждый участник привносит собственное понимание приоритетов, сроков и способов выполнения работы. В таких условиях особую роль играет способность правильно формулировать цели и разбивать их на более простые и понятные этапы, а также эффективно распределять эти этапы между участниками команды. Подобный подход позволяет лучше понимать структуру работы и постепенно двигаться к результату без перегрузки и потери мотивации.

В практике планирования широко используется метод декомпозиции целей. Суть данного подхода заключается в разделении одной большой цели на несколько более мелких задач, выполнение которых постепенно приводит к достижению общего результата. Такой метод помогает сделать сложную цель более понятной и управляемой как для отдельного человека, так и для группы. Когда задача разделена на небольшие этапы, становится проще определить последовательность действий.

Однако на практике процесс декомпозиции целей часто вызывает трудности. Формулировка самой цели обычно не вызывает проблем, но дальнейшее разложение её на конкретные шаги требует определённых навыков планирования и понимания структуры будущей работы. В коллективных проектах

эта сложность усугубляется необходимостью учитывать не только содержание задач, но и зависимости между действиями разных участников, а также поддерживать единое видение последовательности шагов.

Многие пользователи сталкиваются с ситуацией, когда общая цель понятна, но последовательность действий для её достижения остаётся неопределённой. Например, при постановке цели изучить иностранный язык, освоить новую профессию или реализовать личный проект возникает вопрос, с чего именно начать и какие шаги следует выполнять дальше. Аналогичная неопределённость возникает при работе над командными проектами, когда участники могут по-разному представлять себе структуру работ и этапы их выполнения.

Попытки самостоятельно разбить такую цель на этапы могут приводить к слишком общим формулировкам или к пропуску важных промежуточных действий. В результате планирование становится менее эффективным, а сама цель может казаться слишком сложной или недостижимой. Отсутствие чёткой структуры задач часто приводит к откладыванию работы и снижению мотивации.

В настоящее время для планирования задач используются различные инструменты. К ним относятся обычные списки дел, заметки, таблицы, специализированные сервисы управления задачами и другие цифровые решения. Такие инструменты позволяют хранить список задач и отслеживать их выполнение, однако они редко помогают на этапе первоначального формирования структуры цели. Пользователь по-прежнему должен самостоятельно определять последовательность действий и формулировать подзадачи. В результате основной этап планирования остаётся полностью ручным и зависит от опыта пользователя.

Целевая аудитория решений в данной области достаточно широка. К ней относятся студенты, планирующие учебную деятельность и подготовку к экзаменам, начинающие специалисты, осваивающие новые навыки или профессии, разработчики и представители технических профессий, работающие над сложными проектами, а также люди, занимающиеся саморазвитием и личными долгосрочными целями. Значительную часть аудитории составляют

команды, реализующие совместные проекты, где потребность в единой системе координации и обмена опытом планирования особенно высока. Для таких пользователей важно иметь возможность быстро сформировать понятный план действий, который можно корректировать по мере выполнения работы. В условиях коллективной работы дополнительную ценность приобретает возможность не только создавать собственные планы, но и опираться на успешные наработки коллег, адаптируя их под конкретные задачи.

Развитие современных цифровых технологий позволяет создавать инструменты, которые могут помогать пользователю на этапе первоначального структурирования цели, а также поддерживать коллективное взаимодействие в процессе её достижения. Подобные системы способны анализировать текстовое описание задачи и предлагать возможные направления её разложения на более простые этапы, предоставляя пользователю ориентировочную структуру для дальнейшей работы. Дополнительно такие инструменты могут обеспечивать создание команд, совместное использование сформированных планов, механизмы копирования и адаптации структур задач между участниками, а также средства коммуникации для обсуждения хода выполнения.

При этом все формируемые рекомендации рассматриваются не как точные инструкции или готовые решения, а как отправная точка, которую пользователь может дополнять, изменять или удалять в зависимости от собственных задач и условий выполнения работы. В результате система выступает не директивным предписанием, а гибкой средой поддержки, адаптируемой под индивидуальные или командные потребности.

Баланс между автоматизацией и свободой ручного управления позволяет сохранить контроль над процессом планирования. В командной работе аналогичный подход позволяет одному участнику поделиться своей структурой плана, а коллеги – скопировать её, адаптировать под свои задачи и вести собственный учёт выполнения, сохраняя при этом общий контекст совместного проекта и возможность оперативного обсуждения возникающих вопросов.

1.2 Определение функциональных задач пользователей

В процессе проектирования веб-приложения для декомпозиции целей и поддержки командного взаимодействия выполнена идентификация основных решаемых пользователями функциональных задач.

Анализ предметной области показал, что разрабатываемая система является самостоятельным и самодостаточным проектом, в связи с чем архитектурное решение не предусматривает выделения отдельной панели системного администрирования верхнего уровня. Это обусловлено ориентацией приложения на децентрализованную модель управления, при которой каждый пользователь или команда функционируют в рамках собственного изолированного рабочего пространства.

Управление правами доступа и функциональными возможностями полностью реализовано на уровне командных пространств, где административные функции делегированы создателям и руководителям команд.

Такой подход исключает необходимость вмешательства глобального администратора в повседневную деятельность пользователей и снижает сложность сопровождения системы в целом. Система разбита на логические модули, в рамках которых выделены следующие роли пользователей:

– авторизованный пользователь. Основная роль, использующая приложение для личного целеполагания и участия в коллективной работе. Функция включает управление личным пространством и взаимодействие в рамках команд, данная роль является базовой и присваивается каждому зарегистрированному участнику системы по умолчанию;

– администратор команды. Роль, наделенная расширенными правами управления ресурсами и составом конкретной команды. Пользователь, инициировавший создание команды, получает данную роль автоматически. Помимо создания команды, администратор может назначать других участников на эту роль, делегируя часть своих полномочий.

На рисунке 1.1 представлены функции авторизованного пользователя в контексте личного планирования и командного взаимодействия.



Рисунок 1.1 – Диаграмма прецедентов «Авторизованный пользователь»

Пользователь запускает веб-приложение и выполняет вход в систему, используя свои учетные данные. После успешной аутентификации приложение готово к работе в двух основных направлениях: личное планирование и командное взаимодействие. В области личного планирования авторизованный пользователь имеет возможность создавать и редактировать личные цели с применением механизма автоматической декомпозиции для упрощения структурирования сложных задач. Пользователь управляет этапами выполнения и подзадачами, отслеживает прогресс и использует встроенные средства визуализации для наглядного отображения структуры целей и текущего статуса их выполнения.

В рамках командного взаимодействия пользователь может создавать новые команды или вступать в уже существующие. Для обеспечения прозрачности

Система обеспечивает отслеживание персональной статистики выполнения задач и своевременную отправку уведомлений о событиях в рамках целей и команд. На рисунке 1.2 представлена диаграмма прецедентов для роли «Администратор команды».

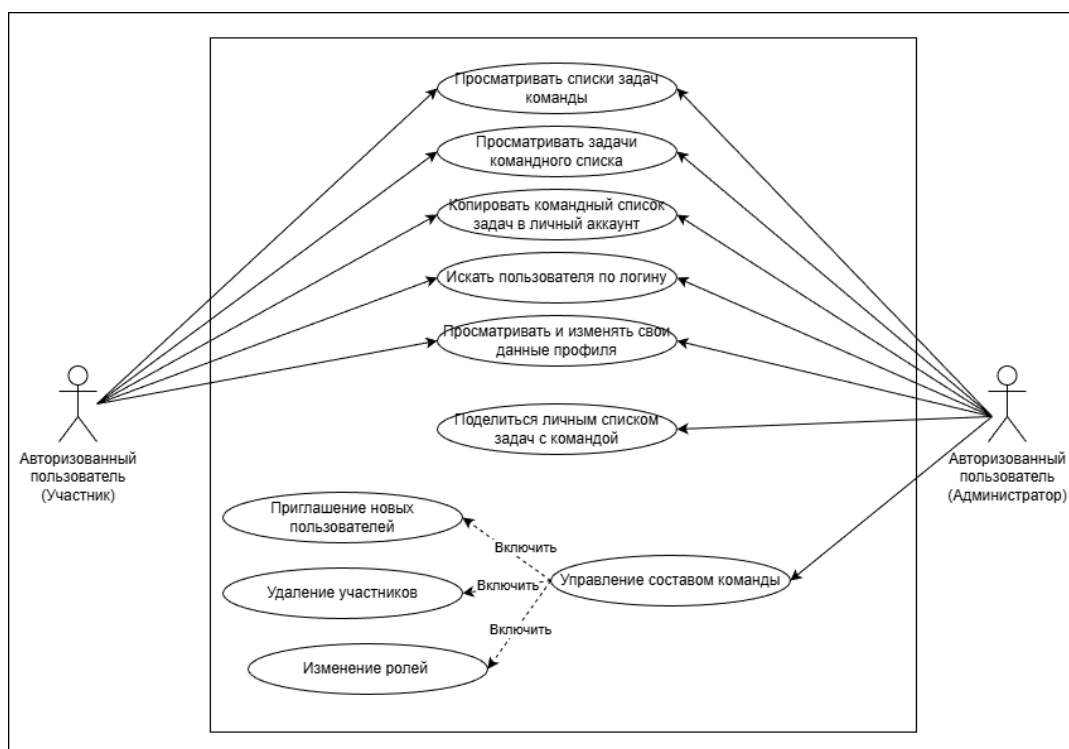


Рисунок 1.1 – Диаграмма прецедентов «Администратор команды»

Пользователь, инициировавший создание новой команды, или участник, назначенный на эту роль, получает доступ к расширенному функционалу управления. Администратор команды отвечает за жизненный цикл группы, включая функции её создания и удаления. В его полномочия входит управление составом участников: приглашение новых членов, удаление пользователей, а также назначение и изменение ролей других участников в пределах данной команды. Разработанное приложение не предусматривает отдельной роли системного администратора. Каждая команда функционирует как автономная

единица, самостоятельно управляя своими ресурсами, данными и составом участников.

Такой подход гарантирует изолированность и автономность групп пользователей, позволяя каждой команде независимо настраивать внутренние процессы и управлять доступом без вмешательства в глобальную конфигурацию системы.

1.3 Анализ аналогов и прототипов

Для разработки системы декомпозиции целей проведён анализ существующих приложений, предназначенных для управления задачами и организации рабочих процессов. Основное внимание уделялось функциональным возможностям, способам представления информации, а также подходам к структурированию целей и задач.

При выборе прототипов рассматривались системы, которые демонстрируют различные подходы к планированию деятельности: от классических списков задач до гибких настраиваемых рабочих пространств и визуальных досок.

В целях выявления положительных сторон выбран Todoist – популярный сервис для управления личными задачами и командными проектами. Данное приложение предоставляет пользователю инструменты для создания проектов, добавления задач и подзадач, установки сроков выполнения и приоритетов. Эффективность структурирования в сервисе достигается за счет поддержки многоуровневой вложенности элементов и распознавания естественного языка при вводе параметров.

Система ориентирована на удобное повседневное планирование и позволяет быстро формировать списки задач с возможностью их фильтрации по различным параметрам, таким как дата, проект или метки. Интерфейс приложения обеспечивает наглядное представление текущих задач и позволяет отслеживать прогресс выполнения. Наличие механизмов автоматического формирования структуры достижения целей, совместной работы и встроенной коммуникации

делает систему актуальной для использования как в образовательной, так и в профессиональной сфере. Синхронизация данных в реальном времени обеспечивает непрерывность рабочего процесса.

На рисунках 1.4 и 1.5 представлены основные элементы интерфейса системы.

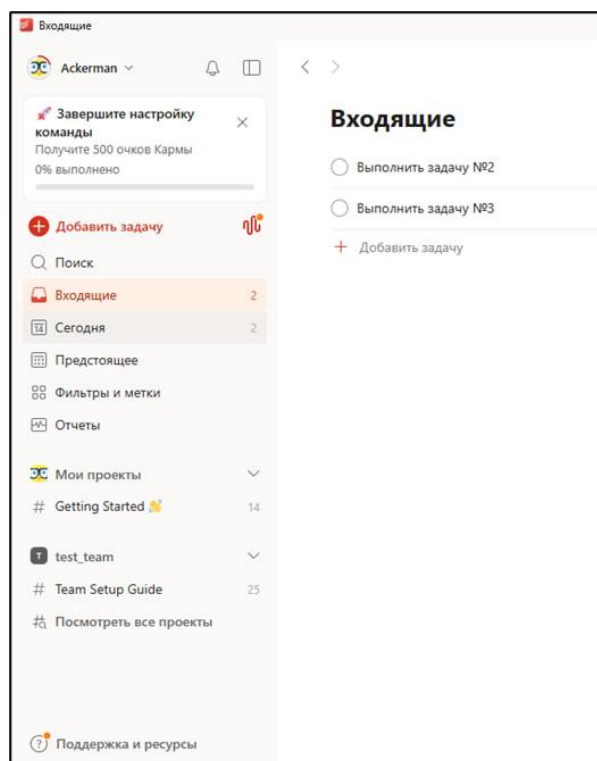


Рисунок 1.4 – Главная страница списка задач «Todoist»

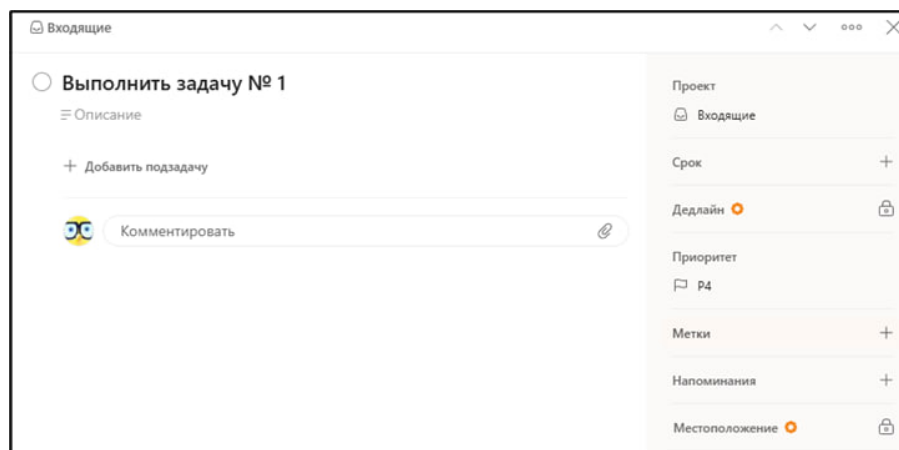


Рисунок 1.5 – Детальная страница задачи в «Todoist»

Для более наглядного анализа и систематизации полученных данных основные параметры исследованного сервиса сгруппированы по ключевым

критериям оценки программного обеспечения. Обобщенные результаты этого анализа, отражающие эксплуатационные и технологические особенности платформы, представлены ниже.

Ключевые характеристики системы в обобщённом виде, представленные на таблице 1.1.

Таблица 1.1 – Характеристика системы «Todoist»

Категория	Описание
Функциональность	Поддержка проектов, задач и подзадач, приоритетов, напоминаний и меток
Удобство	Интуитивный интерфейс, быстрое добавление и структурирование задач
Безопасность	Использование облачного хранения данных и механизмов авторизации
Стабильность	Синхронизация данных между устройствами, стабильная работа сервиса
Доступность	Веб-версия и мобильные приложения, различные тарифные планы

Для реализации более сложных подходов к планированию целесообразно рассмотреть платформу Notion, которая позиционируется как универсальная экосистема для организации информации. Пространство Notion эффективно объединяет в себе функциональность динамических баз данных, текстового редактора и инструментов управления проектами.

Одной из ключевых особенностей Notion является высокая гибкость настройки. Пользователь может создавать собственные структуры данных, связывать таблицы между собой и использовать различные представления: таблицы, доски, списки и календари. Это позволяет реализовывать сложные системы декомпозиции целей и адаптировать рабочее пространство под конкретные задачи.

Дополнительной особенностью платформы является наличие встроенного ИИ-ассистента (Искусственный интеллект), который помогает в формировании и структурировании задач. Также система поддерживает совместную работу, управление доступом и хранение истории изменений.

Важным элементом платформы является возможность использования готовых шаблонов, которые позволяют ускорить процесс организации рабочего пространства и стандартизировать подход к планированию задач. Пользователь может как применять существующие шаблоны, так и разрабатывать собственные, адаптированные под индивидуальные или командные потребности.

На рисунках 1.6 и 1.7 представлены элементы интерфейса системы.

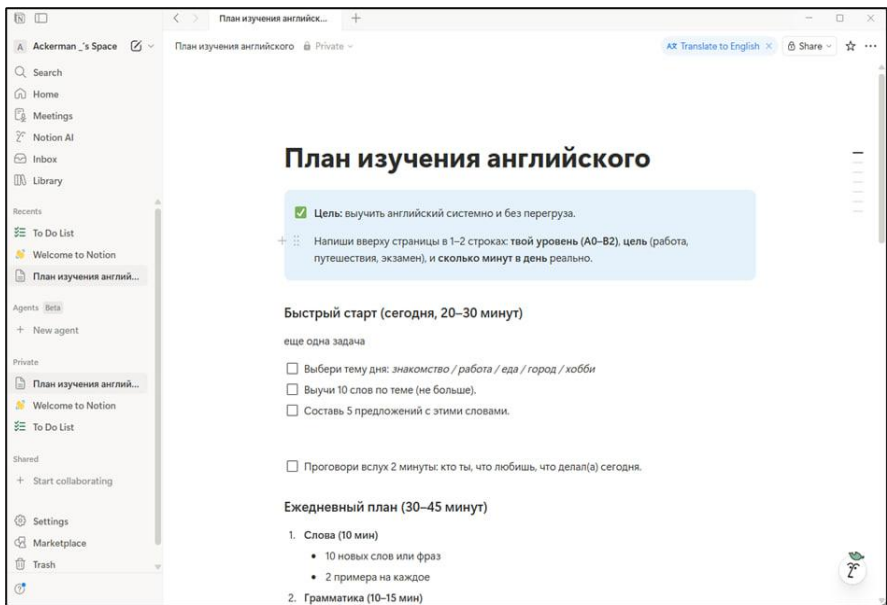


Рисунок 1.6 – Главная страница списка задач в «Notion»

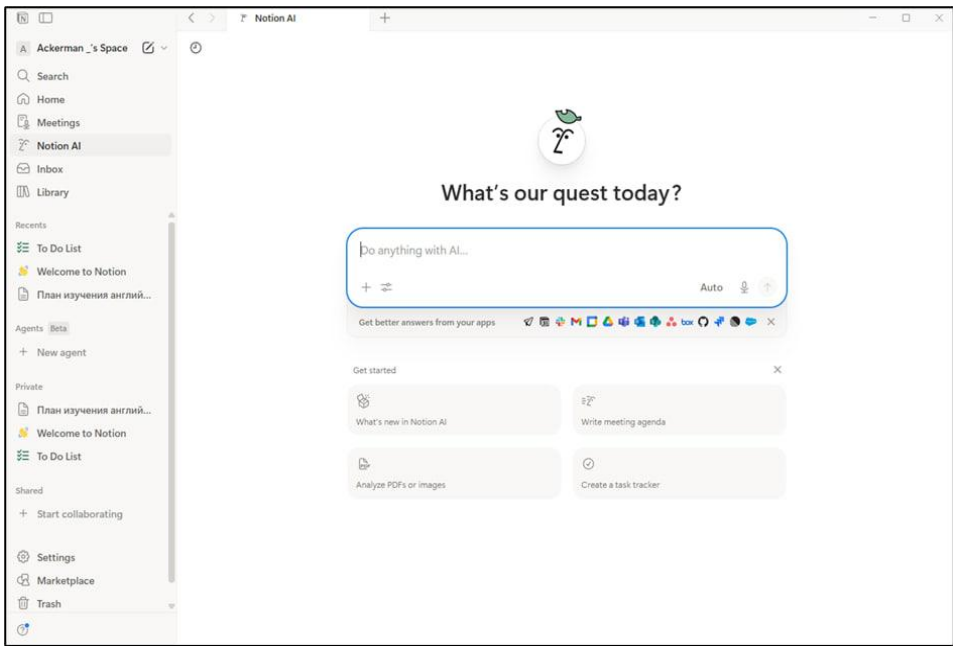


Рисунок 1.7 –Страница взаимодействия с ИИ в «Notion»

Для полноты анализа функционал и технические параметры исследуемой системы систематизированы. Это позволяет наглядно оценить практические аспекты внедрения платформы и сопоставить её свойства с требованиями к разрабатываемому решению.

Ключевые характеристики системы в обобщённом виде, представленные на таблице 1.2.

Таблица 1.2 – Характеристика системы «Notion»

Категория	Описание
Функциональность	Гибкая настройка структур данных, шаблоны, различные способы представления
Удобство	Поддержка таблиц, досок, календарей и пользовательских интерфейсов
Безопасность	Разграничение прав доступа и управление данными
Стабильность	Регулярные обновления и синхронизация между устройствами
Доступность	Кроссплатформенность и наличие различных вариантов использования

Отдельное внимание стоит уделить платформе Yougile, в основе которой лежит классический подход к визуализации потока задач с помощью досок и карточек. Такой формат позволяет наглядно отображать этапы выполнения работы и гибко реагировать на изменения в проекте.

В данной системе задачи организуются в виде карточек, размещённых на досках, которые отражают этапы выполнения работы. Пользователь может перемещать задачи между этапами, что обеспечивает наглядное отслеживание прогресса.

Каждая карточка может содержать подробную информацию о задаче, включая описание, комментарии и вложения, что обеспечивает централизованное хранение данных по проекту. Это позволяет участникам команды взаимодействовать в рамках единого рабочего пространства без необходимости использования сторонних инструментов. Дополнительным преимуществом платформы является встроенный мессенджер в каждой задаче, который превращает обсуждение рабочих вопросов в структурированные чаты, исключая

потерю важной информации.

В таблице 1.3 представлены ключевые характеристики системы в обобщенном виде.

Таблица 1.3 – Характеристика системы «Yougile»

Категория	Описание
Функциональность	Визуальные доски, карточки задач, управление этапами выполнения
Удобство	Наглядное представление задач и простая работа с ними
Безопасность	Управление доступом пользователей и проектных данных
Стабильность	Поддержка работы через веб и мобильные устройства
Доступность	Возможность использования в командах различного масштаба

На рисунке 1.8 представлен пример интерфейса системы.

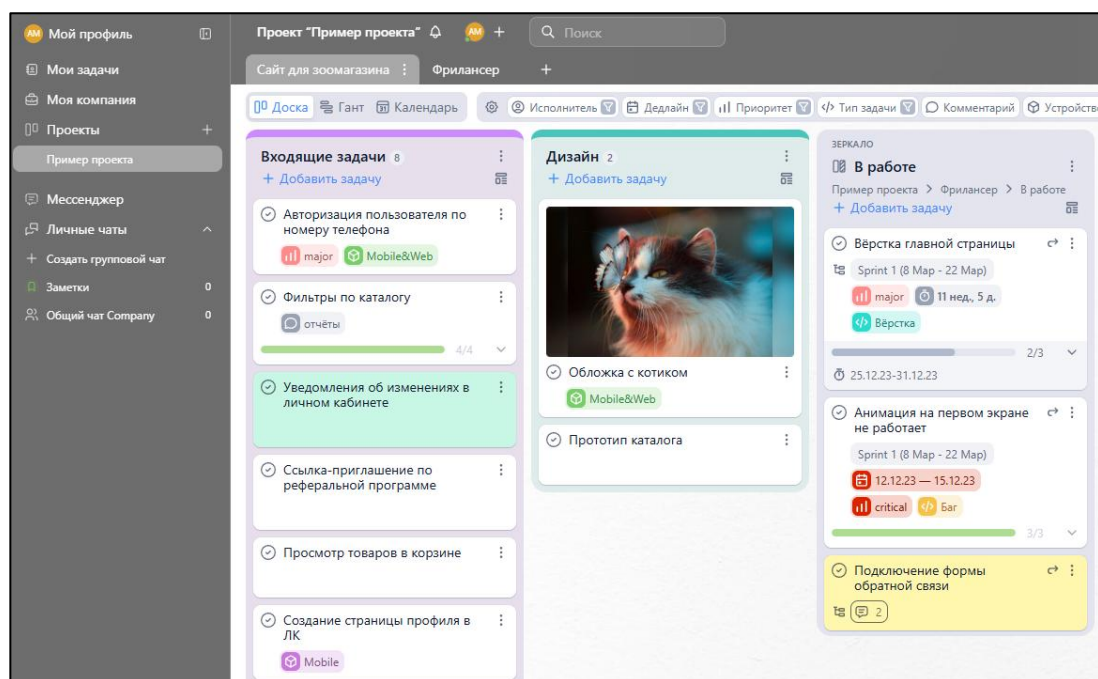


Рисунок 1.8 – Главная страница доски с списками задач в «Yougile»

Проведённый анализ показал, что современные системы управления задачами предлагают разнообразные инструменты для планирования, визуализации и организации деятельности. Они позволяют структурировать задачи, отслеживать их выполнение и обеспечивать совместную работу пользователей. Для обобщения результатов анализа представим сравнительную характеристику рассматриваемых систем по ключевым функциям.

В таблице 1.4 приведено сопоставление сервисов, отражающее их основные возможности

Таблица 1.4 – Общий анализ сервисов

Функция	Todoist	Notion	Yougile
Работа с задачами	Да	Да	Да
Визуальное представление	Да	Да	Да
Совместная работа	Да	Да	Да
Управление доступом	Да	Да	Да
Подзадачи, декомпозиция	Да	Да	Да
Встроенный ИИ-ассистент	Нет	Да	Нет
Учет времени выполнения	Нет	Нет	Да
Наличие готовых шаблонов	Да	Да	Нет
Возможность изменения дизайна	Нет	Да	Да
Импорт и экспорт данных	Да	Нет	Нет

1.4 Проектирование информационной системы

В процессе проектирования выделены следующие основные функции системы декомпозиции целей:

- вход в систему с использованием учетной записи пользователя;
- формулирование цели и передача текстового описания в сервис искусственного интеллекта;
- автоматическая генерация структурированного плана на основе введённой цели с разделением на задачи;
- просмотр списка личных списков задач и детальный просмотр задач внутри выбранного плана;
- управление задачами (создание, редактирование, удаление, изменение порядка следования, отметка о выполнении);
- совместная работа с целями в рамках команды, включая предоставление доступа к личному списку задач и просмотр командных списков;
- копирование командного списка задач в личный аккаунт пользователя;
- ведение и просмотр истории взаимодействия с сервисом ИИ;

– поиск пользователей по имени и управление базовыми настройками профиля.

Приведённый перечень функций отражает ключевые сценарии взаимодействия пользователя с приложением и формирует основу для дальнейшей детализации программных модулей. Каждая из указанных операций предполагает реализацию соответствующего интерфейсного компонента и серверной логики обработки данных.

Для наглядного отображения функциональных возможностей и взаимодействия компонентов системы, разработаны графические схемы, иллюстрирующие последовательность выполнения ключевых операций и обмен между различными частями приложения.

На рисунке 1.7 представлена диаграмма последовательности «Генерация списка задач с использованием ИИ»

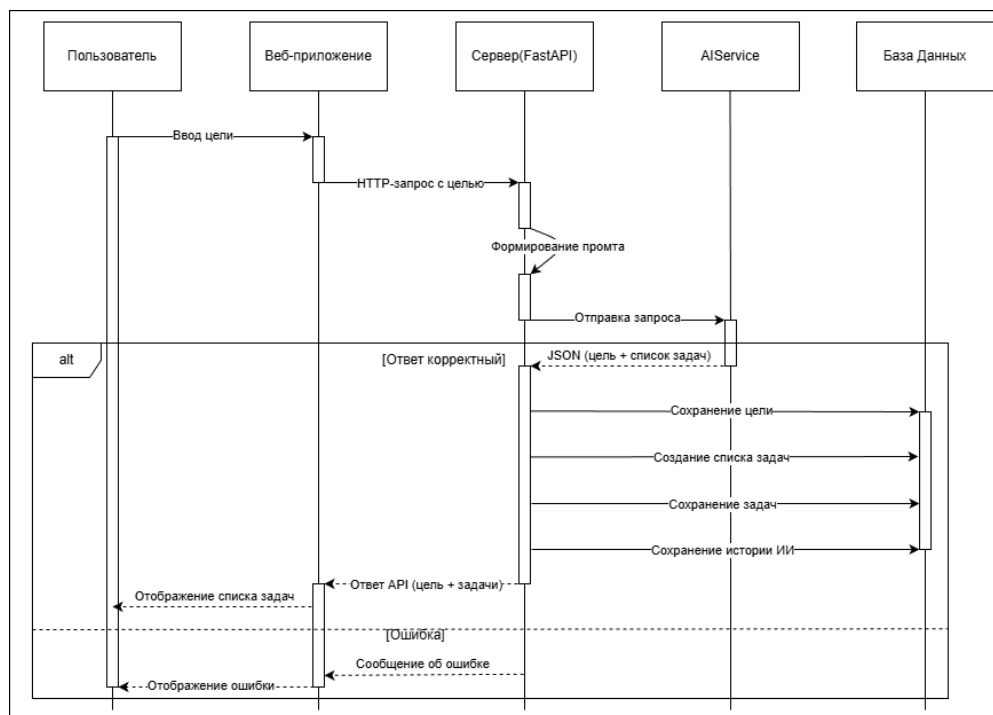


Рисунок 1.7 – Диаграмма последовательности для функции генерации списка задач с использованием ИИ»

На диаграмме показано взаимодействие основных участников процесса: пользователя, клиентского веб-приложения, сервера и ИИ-сервиса. Пользователь

формулирует цель и отправляет запрос через интерфейс веб-клиента. Клиентское приложение формирует запрос к серверу, представленному в виде API, который, в свою очередь, обращается к сервису с подготовленным руководством общения с пользователем в виде набора текстовых правил.

Сервис обрабатывает полученный текст цели и генерирует структурированный перечень этапов её достижения. Сервер принимает ответ от сервиса и преобразует в формат, пригодный для хранения в базе данных. После получения структурированного ответа сервис сохраняет данные о цели, списке задач и истории запроса в базе данных и возвращает результат клиентскому приложению. В итоге пользователю отображается сформированный план, доступный для дальнейшего редактирования.

Диаграмма деятельности для вывода задач в списке представлена на рисунке 1.8 и отражает последовательность действий при выводе задач, принадлежавших одному списку задач

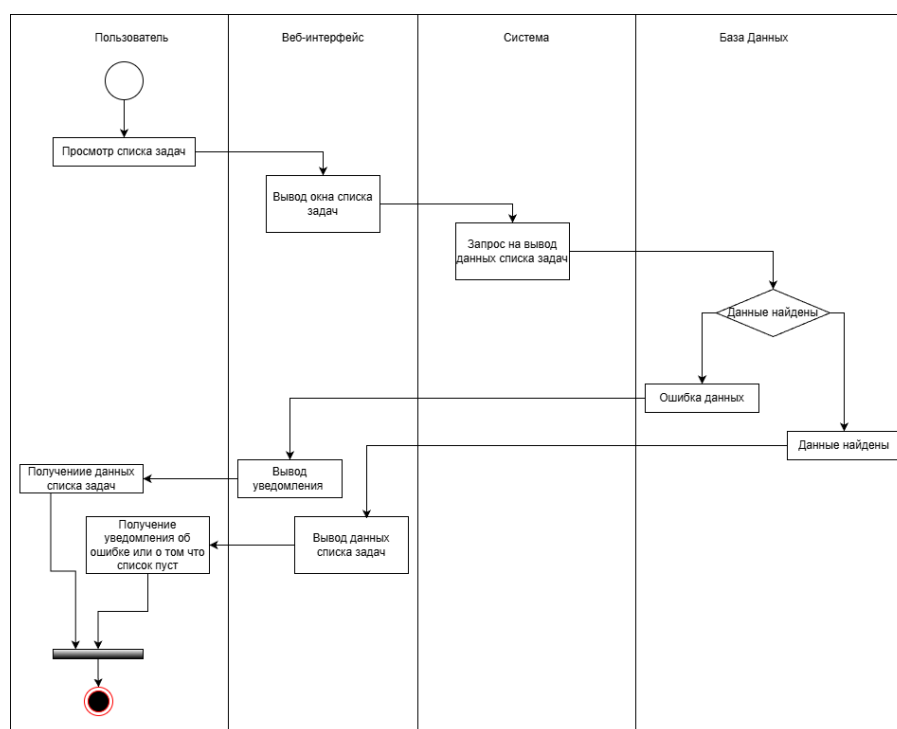


Рисунок 1.8 – Диаграмма деятельности для функции вывода задач списка»

Пользователь авторизуется в системе, выбирает нужный список задач, Система отправляет запрос на вывод данных в базе, обрабатывает результат и

выводит отображение задач в интерфейсе. Перед отправкой запроса серверная часть осуществляет верификацию прав пользователя на доступ к выбранному информационному ресурсу. При этом клиентская часть приложения формирует структурированное представление полученной информации, группируя задачи в соответствии с их статусом выполнения и заданным порядком следования.

В случае сбоя при обращении к базе данных система инициализирует протокол обработки исключений и выводит соответствующее уведомление. При успешном завершении операции пользователь видит задачи, принадлежавшие списку, что позволяет отслеживать какие задачи следует выполнить пользователю для достижения цели. Наличие актуального перечня задач в едином интерфейсе способствует формированию целостного представления о текущем состоянии.

На рисунке 1.9 представлена диаграмма разворачивания системы, демонстрирующая физическое расположение и взаимодействие основных компонентов.

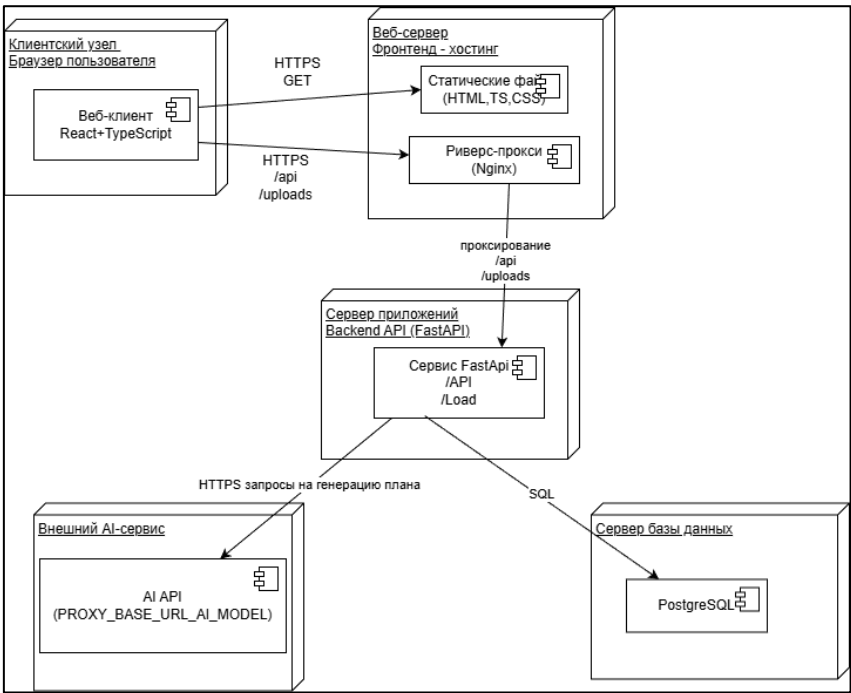


Рисунок 1.9 – Диаграмма разворачивания системы

На диаграмме выделены клиентский узел с веб-приложением, серверный узел с сервисом на FastAPI, узел базы данных, а также внешний узел ИИ-сервиса.

Показаны сетевые связи между клиентом и API, взаимодействие сервиса с базой данных при чтении и записи информации о пользователях, целях и задачах, а также обмен данными между сервисом и ИИ-сервисом при генерации списка задач. Такое представление позволяет наглядно показать архитектуру системы и распределение ответственности между её компонентами.

1.5 Анализ программных ресурсов, необходимых в работе

Для разработки серверной части системы декомпозиции целей планируется использовать фреймворк FastAPI, который представляет собой современный инструмент для создания веб-API на языке программирования Python.

Выбор обусловлен тем, что он позволяет описывать маршруты и структуры данных с помощью аннотаций типов. FastAPI поддерживает асинхронный ввод-вывод, что дает возможность эффективно работать с большим количеством одновременных запросов. Это имеет значение в контексте проекта, поскольку предполагается обращение к внешнему сервису искусственного интеллекта, а также выполнение операций чтения и записи в базе данных.

Одним из важных особенностей является наличие встроенной автоматической генерации документации в формате OpenAPI и Swagger. Это позволяет наглядно представлять доступные точки взаимодействия с сервером и форматы передаваемых данных.

В рамках разработки необходима поддержка CORS (Cross-Origin Resource Sharing – механизм, позволяющий веб-приложениям выполнять запросы к серверу, расположенному на другом домене), что необходимо для корректной работы браузерного клиента.

В качестве основного языка программирования серверной части выбран Python. Это связано с тем, что язык обладает простым и читаемым синтаксисом, что облегчает разработку и сопровождение проекта. Python имеет развитую экосистему библиотек, которые могут быть использованы для реализации аутентификации пользователей, взаимодействия с базой данных, обработки

HTTP-запросов и интеграции с внешними API. В рамках проекта Python будет применяться для описания моделей данных, реализации бизнес-логики системы, а также для взаимодействия с внешним ИИ-сервисом.

Для хранения данных планируется использовать реляционную систему управления базами данных PostgreSQL. Этот выбор объясняется тем, что PostgreSQL обеспечивает надежное хранение структурированных данных и поддерживает выполнение сложных запросов и транзакций.

В контексте системы декомпозиции целей это важно, так как необходимо хранить информацию о пользователях, целях, списках задач, командах и правах доступа, а также историю взаимодействия с искусственным интеллектом. Использование реляционной модели позволяет обеспечить целостность данных и логическую связанность между различными сущностями системы. Взаимодействие с базой данных предполагается реализовать с использованием ORM (Object-Relational Mapping – отображение объектов в реляционные структуры), что позволит работать с данными в объектно-ориентированном стиле и упростит разработку. Также планируется применение асинхронного драйвера для работы с базой данных, что позволит избежать блокировок при выполнении длительных операций и сохранить отзывчивость системы.

Для разработки клиентской части системы планируется использовать TypeScript, React и Vite. TypeScript представляет собой расширение языка JavaScript с поддержкой статической типизации. Его использование позволяет выявлять ошибки на этапе компиляции и делает структуру проекта более понятной за счет явного описания типов данных. Это особенно полезно при разработке сложного интерфейса с большим количеством компонентов и взаимодействий между ними.

В качестве основного инструмента для построения пользовательского интерфейса выбран React – библиотека для создания веб-приложений на основе компонентного подхода. Такой подход позволяет разбивать интерфейс на независимые части с возможностью многократного использования в разных

частях проекта, что позволяет минимизировать загроуженность в местах, где один элемент необходимо использовать несколько раз.

В рамках системы предполагается реализация страниц аутентификации, управления целями и задачами, взаимодействия с искусственным интеллектом. React позволяет гибко управлять состоянием интерфейса и обновлять отображение данных в зависимости от действий пользователя.

Для сборки и локальной разработки дизайна выбран инструмент Vite. Он обеспечивает быструю загрузку проекта и мгновенное обновление изменений в браузере за счет механизма горячей перезагрузки модулей. Это позволяет быстрее тестировать изменения. Для стилизации пользовательского интерфейса планируется использовать Tailwind CSS – утилитарный CSS-фреймворк (Cascading Style Sheets – каскадные таблицы стилей). Он предоставляет набор готовых классов для управления внешним видом элементов, таких как отступы, цвета, размеры и расположение. Такой подход позволяет быстро создавать интерфейс без необходимости писать большое количество пользовательских стилей и делает процесс разработки более гибким.

Взаимодействие с внешним сервисом искусственного интеллекта предполагается реализовать через отдельный сервисный слой на стороне сервера. Для этого будет использоваться HTTP-клиент с поддержкой асинхронных запросов. Сервер будет отправлять текст цели и дополнительные параметры в ИИ-сервис и получать ответ в формате JSON. Полученные данные будут проходить валидацию, преобразовываться во внутренние структуры и сохраняться в базе данных.

Такой подход позволяет разделить ответственность между компонентами системы: серверная часть отвечает за обработку данных и бизнес-логику, а внешний сервис – за генерацию содержимого задач.

Таким образом, выбранный стек технологий, включающий FastAPI и Python для серверной части, PostgreSQL для хранения данных, а также TypeScript, React, Vite и Tailwind CSS для клиентского приложения.

2 Технологический раздел

2.1 Структура базы данных

При анализе предметной области сервиса «декомпозиции целей с ИИ» выделены следующие сущности:

– пользователи (Users): содержит информацию об учетной записи пользователя: уникальный идентификатор, логин, имя, фамилию, отчество, адрес электронной почты, хэш пароля для аутентификации, ссылку на фотографию профиля, а также даты создания и последнего обновления записи;

– цели (Goals): содержит информацию о цели пользователя, заголовок, описание, дату создания.

– списки задач (Roadmaps): содержит информацию о плане достижения цели, даты создания, обновления, ссылку на цель и принадлежность команде. Списки включают задачи и могут иметь записи доступа и копирования;

– задачи (Tasks): содержит информацию о шагах списка, заголовок, описание, порядок выполнения, статус выполнения, дату выполнения, дату создания, а также ссылку на список задач;

– команды (Teams): содержит информацию о рабочих группах пользователей, внутри которых можно совместно работать с списками задач;

– участники команд (TeamMember): содержит информацию о принадлежности пользователя к команде, а также параметры участия, такие как роль;

– роли в команде (TeamRole): содержит информацию о ролях, уровнях прав участника в рамках команды;

– ссылки доступа в команду (TeamAccessLink): содержит информацию о пригласительных ссылках для присоединения к команде;

– доступ к спискам задач (RoadmapAccess): содержит информацию о правах доступа пользователя к списку целей. Это позволяет делиться списками между участниками команды;

- копирование списка задач (RoadmapCopy): содержит информацию о копировании, связи оригинала и новой копии;
- чаты (Chat): содержит информацию о чатах, к которым подключаются пользователи;
- участники чатов (ChatParticipant): содержит информацию о составе чата;
- сообщения (Message): содержит информацию о сообщениях пользователей в чатах;
- ИИ диалоги (AIConversation): содержит информацию о сессиях общения пользователя с ИИ;
- сообщения ИИ диалога (AIMessage): содержит информацию о сообщениях;
- роли сообщений ИИ (AIMessageRole): модель, содержащая информацию о роли сообщения в диалоге.

ER-диаграмма связей между сущностями вынесена в приложение А, словарь данных с типами полей и ограничениями представлен в таблице приложения Б.

2.2 Разработка интерфейса программного продукта

Дизайн веб-приложения спроектирован в графическом редакторе Figma. В процессе проектирования определены основные экранные формы и визуальные сценарии взаимодействия с системой. Особое внимание уделено эргономике интерфейса, минимализму и логичности пространственного расположения элементов, что обеспечивает визуальный комфорт и снижение когнитивной нагрузки при длительной работе с программным продуктом.

При формировании интерфейса использована цветовая палитра с преобладанием фиолетовых оттенков на темном фоне. Данное решение позволяет акцентировать внимание на кнопках и заголовках без создания избыточной яркости. Сочетание контрастных светлых и темных тонов формирует ощущение технологичности и обеспечивает визуальную доступность элементов управления.

Создание детальных интерактивных прототипов позволило на раннем этапе смоделировать пользовательские пути и оптимизировать переходы между разделами.

В структуру интерфейса вошли следующие основные экраны:

- аутентификации;
- ИИ-помощника;
- управления списками задач;
- профиля;
- обмена сообщениями;
- отображения команд.

Для веб-приложения также разработан логотип, представленный на рисунке 2.1.



Рисунок 2.1 – Логотип веб-приложения

Графическое решение логотипа сочетает в себе стилизованные изображения планшета с отмеченными задачами и мишени со стрелой в центре, что тематически символизирует планирование и точное достижение результатов. Данный элемент интегрирован во все основные страницы приложения в качестве сквозного идентификатора стиля, обеспечивая целостность визуального восприятия и узнаваемость интерфейса. Экранные формы спроектированы с

учетом современных требований к композиции веб-ресурсов.

Для обеспечения безопасного доступа к функционалу системы разработан экран аутентификации, представленный на рисунке 2.2.

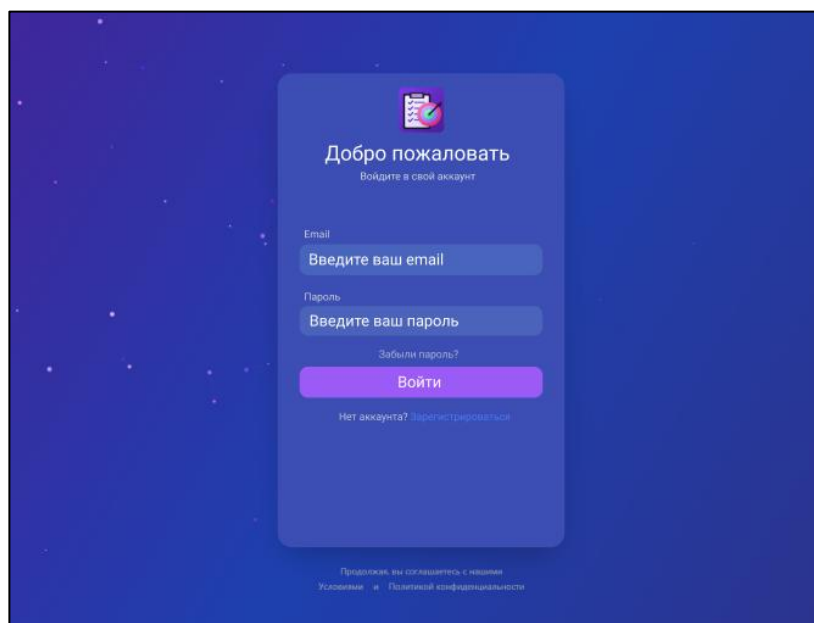


Рисунок 2.2 – Экран аутентификации

Экран входа реализован с учётом требований удобства и простоты использования. Пользователь может выполнить вход в систему с помощью адреса электронной почты и пароля. Центральное положение формы аутентификации фокусирует внимание пользователя на процессе ввода данных, минимизируя лишние визуальные отвлекающие факторы.

Интерфейс оформлен в темных тонах на фоне плавного градиента, что делает страницу визуально привлекательной и современной. В верхней части формы расположен фирменный логотип приложения, подчеркивающий стилевое единство программного продукта.

Поля ввода снабжены контрастными подсказками, которые упрощают взаимодействие с интерфейсом на интуитивном уровне. Дополнительно предусмотрены ссылки для регистрации нового пользователя и восстановления пароля, что обеспечивает поддержку основных сценариев авторизации. Яркая фиолетовая кнопка действия «Войти» выступает главным композиционным

акцентом экранной формы, побуждая к завершению целевого действия.

После успешной аутентификации пользователь получает доступ к персональному профилю, экран которого представлен на рисунке 2.3.

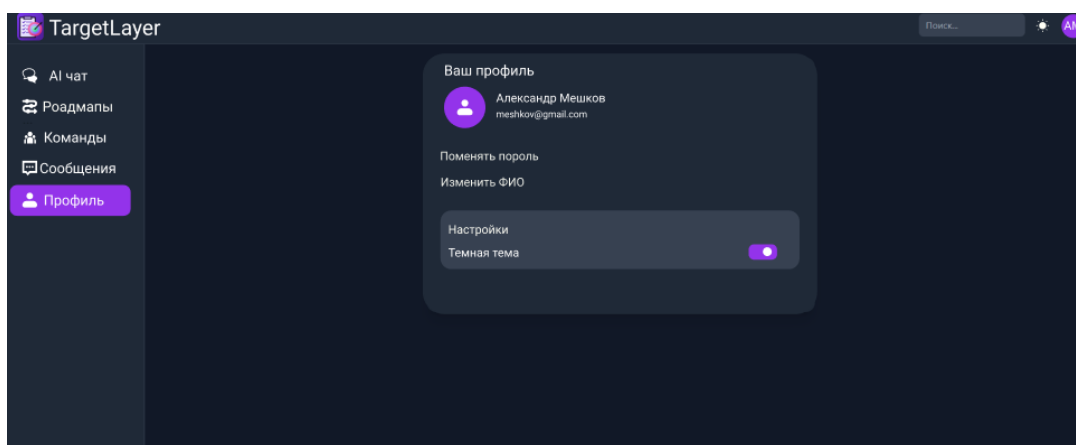


Рисунок 2.3 – Экран профиля пользователя

Экран профиля содержит основную информацию об аккаунте пользователя. Дополнительно реализована возможность переключения между светлой и тёмной темой оформления интерфейса. Дизайн экрана выполнен в лаконичном стиле, благодаря чему вся необходимая информация доступна пользователю с первого взгляда.

Для использования ИИ-помощника разработан экран необходимого чата, представленный на рисунке 2.4.

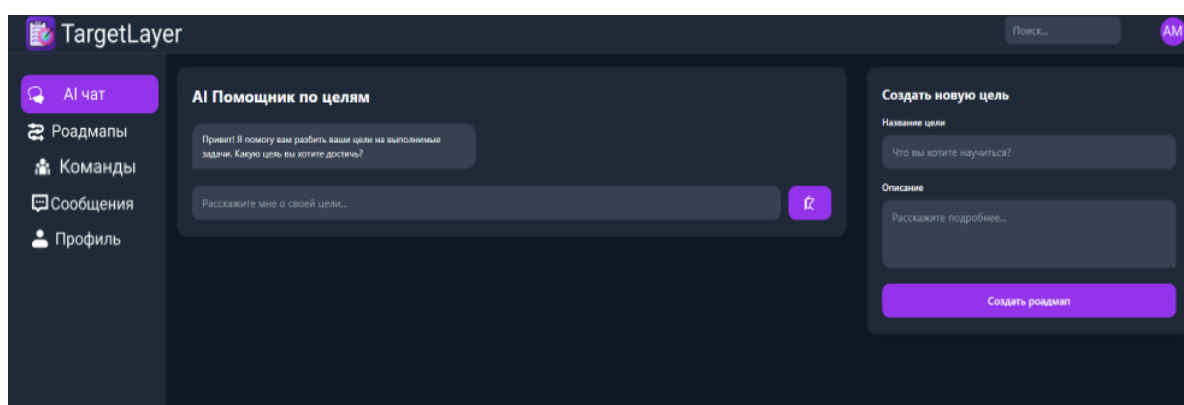


Рисунок 2.4 – Экран с ИИ-чатом и созданием целей

На данном экране пользователь взаимодействует с ИИ-помощником, который помогает формировать и декомпозировать цели на отдельные задачи.

Интерфейс экрана разделён на две основные области: чат с ИИ-помощником и форма создания новой цели.

В форме предусмотрены поля для ввода названия и описания цели, а также кнопка «Создать список задач». Минималистичный дизайн экрана позволяет пользователю сосредоточиться на взаимодействии с системой без отвлекающих элементов.

Для удобного взаимодействия с задачами разработана страница управления списками задач, представленная на рисунке 2.5.

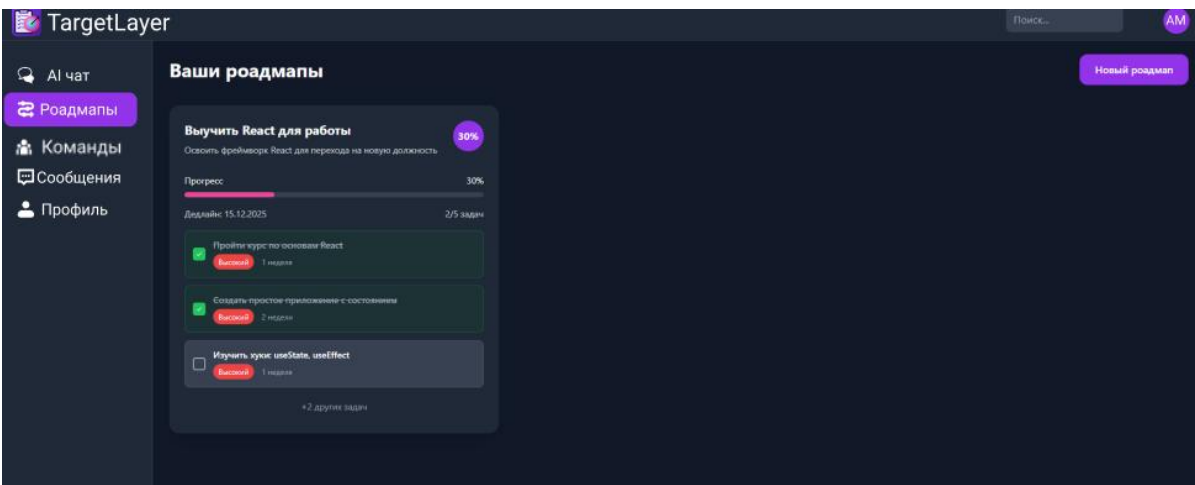


Рисунок 2.5 – Экран управления списками задач

На данной странице визуализированы структурированные списки целей с декомпозицией на отдельные подзадачи. Архитектура макета включает в себя графические маркеры статуса выполнения для оперативного мониторинга прогресса, а также выделенный элемент для добавления новых списков. Ключевые компоненты интерактивного взаимодействия и индикаторы снабжены цветовым выделением, что повышает интуитивность восприятия интерфейсной логики. Наличие свободного пространства между информационными блоками предотвращает визуальное перегруженные интерфейса при отображении большого количества подзадач.

Помимо индивидуальной работы с задачами, макет предоставляет элементы, демонстрирующие возможность взаимодействия пользователей в

рамках команд. Для этого разработан отдельный экран команд, представленный на рисунке 2.6.

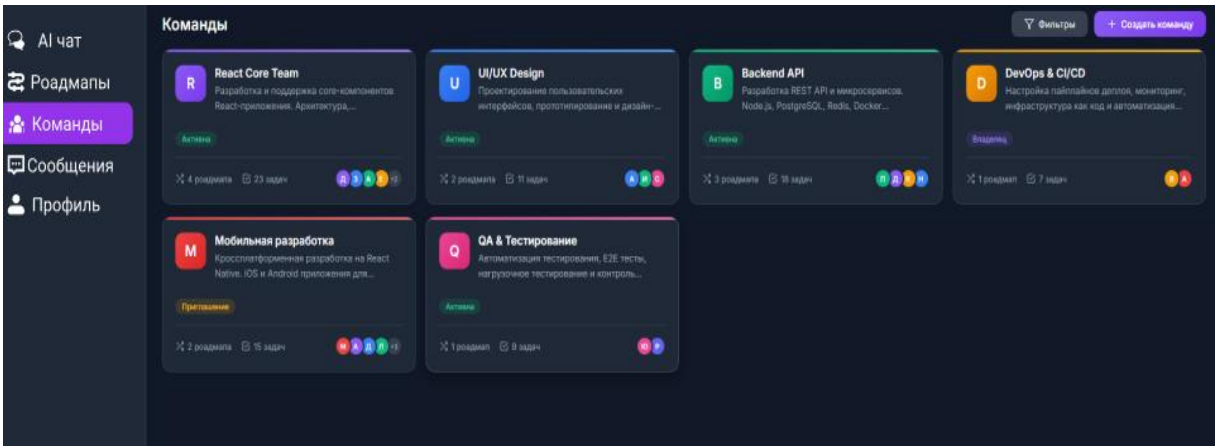


Рисунок 2.6 – Экран команд пользователя

На экране команд отображаются команды, участником которых является пользователь. Для каждой команды выводится её название.

Для общения между пользователями разработан экран сообщений, представленный на рисунке 2.7.

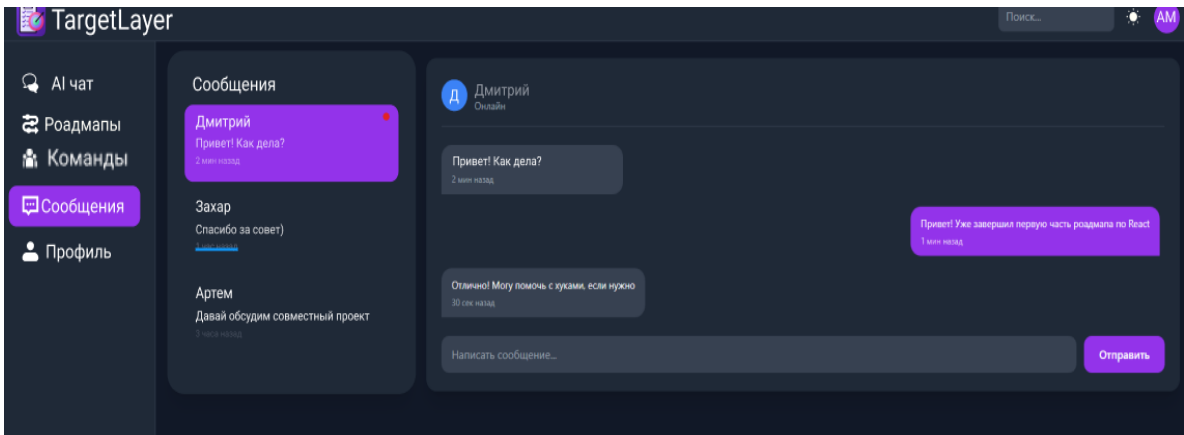


Рисунок 2.7 – Экран сообщений между пользователями

На экране сообщений отображаются элементы для удобного перехода между необходимыми чатами, а также строка ввода сообщений и кнопку для отправления сообщений.

Таким образом, разработанные экраны интерфейса обеспечивают удобное взаимодействие пользователя с системой, поддерживают основные

пользовательские сценарии и создают единый современный стиль.

2.3 Разработка структуры программного продукта

Так как система представляет собой веб-приложение, составлена функциональная схема веб-клиента, включающая публичную и внутреннюю части. Публичная часть, представленная на рисунке 2.1, доступна пользователю до авторизации и содержит окна «Авторизация», «Регистрация» и «Восстановление пароля». Дополнительно в неё включены окно «Ошибочного пути» (страница 404) и окно «Краткой информации» о системе. Пользователь может создать учётную запись, войти в систему или восстановить утерянный пароль, а также ознакомиться с основными возможностями сервиса.

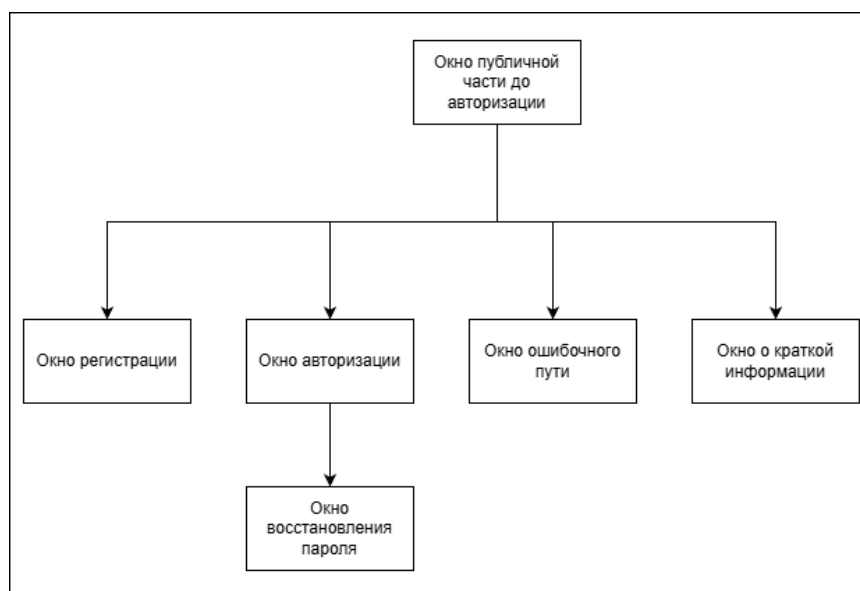


Рисунок 2.8 – Функциональная схема публичной части веб-приложения

После успешной авторизации пользователю становится доступен основной функционал системы. Внутренняя часть включает разделы: «Профиль», «Смена пароля», «Списки задач», «Просмотр конкретного списка и его задач», «Команды», «Списки задач команды», «Чаты», «Просмотр конкретного чата», «Поиск пользователей», а также интерфейс общения с ИИ-чатом и историю ИИ-диалогов.

В разделе «Профиль» пользователь просматривает и редактирует свои

данные, а также может перейти к окну «Смена пароля» для обновления учетных данных. Раздел «Списки задач» отображает созданные пользователем планы и позволяет перейти к детальному просмотру конкретного списка, где выполняется управление задачами, их порядком и статусом выполнения.

В разделе «Команды» пользователь создает команды и переходит к их данным, а в окне «Списки задач команды» организуется совместная работа над планами. Раздел «Чаты» обеспечивает общение: пользователь выбирает диалог и переходит в окно конкретного чата для просмотра сообщений и отправки новых. Раздел «Поиск пользователей» служит для нахождения других участников системы. Отдельный блок составляет интерфейс ИИ-чата, позволяющий вести диалог с ИИ, получать начальный список задач и просматривать историю предыдущих обращений. На рисунке 2.2 изображена функциональная схема внутренней части веб-приложения.

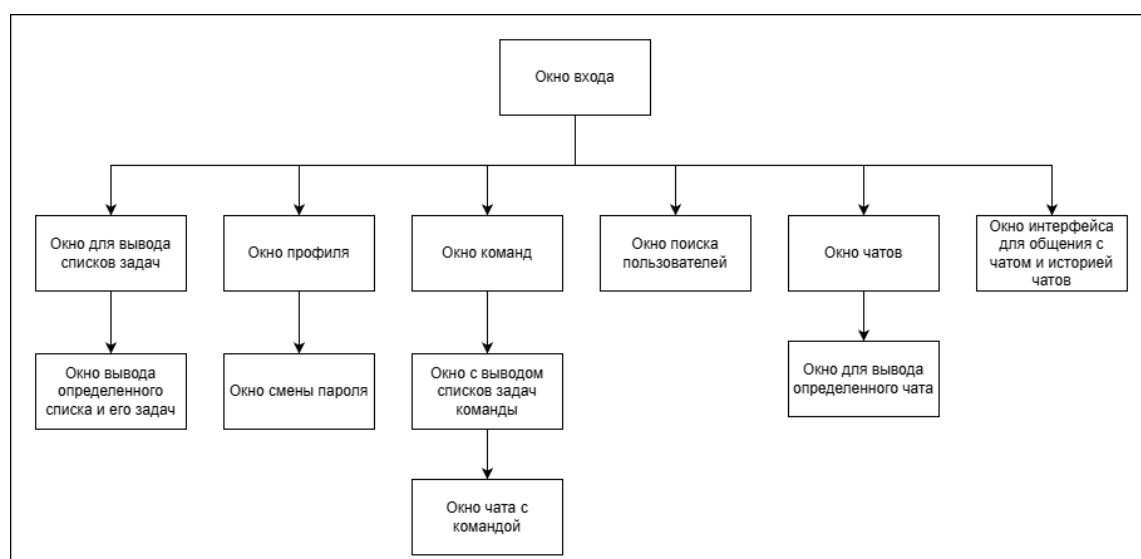


Рисунок 2.9 – Функциональная схема внутренней части веб-приложения

2.4 Описание процедур и функций

В процессе разработки веб-приложения реализована совокупность функций и процедур, обеспечивающих формирование списков задач, управление задачами, контроль доступа пользователей и взаимодействие с сервисом искусственного интеллекта. Архитектура приложения построена по принципу разделения

ответственности между компонентами, благодаря чему обеспечивается масштабируемость системы, удобство сопровождения и устойчивость работы приложения при одновременной работе нескольких пользователей.

Одной из ключевых функций системы является автоматическое формирование списка задач с использованием искусственного интеллекта. Для реализации данной возможности разработан сервис AIRoadmapService, обеспечивающий взаимодействие серверной части приложения с внешним ИИ API. Основной задачей сервиса является обработка пользовательского запроса, формирование контекста диалога и получение структурированного набора задач для будущего списка.

Перед отправкой запроса система формирует массив сообщений, содержащий системный свод правил, историю взаимодействия пользователя и текущее сообщение. Дополнительно задаются параметры генерации текста, включая используемую модель и ограничения на размер ответа. На рисунке 2.10 представлен фрагмент формирования запроса к сервису искусственного интеллекта.

```
ai_chat_roadmap.py

messages = [{"role": "system", "content": system_prompt}]
if conversation_history:
    messages.extend(conversation_history)
    messages.append({"role": "user", "content": user_message})

payload = {
    "model": self.model,
    "messages": messages,
    "temperature": GENERATION_CONFIG["temperature"],
    "max_tokens": GENERATION_CONFIG["max_tokens"],
}

async with httpx.AsyncClient(Timeout=self.timeout) as client:
    resp = await client.post(url, headers=headers, json=payload)
```

Рисунок 2.10 – Формирование запроса к сервису ИИ

После формирования объекта запроса выполняется асинхронное обращение к ИИ сервису с использованием библиотеки httpx. Применение асинхронных

запросов позволяет избежать блокировки серверного потока во время ожидания ответа от внешнего сервиса. После получения ответа система извлекает содержимое сообщения и преобразует его в JSON-структуру, содержащую название цели и список задач. На рисунке 2.11 представлен фрагмент формирования контекста.

```
ai_chat_roadmap.py

context_section += f"Название цели: {roadmap_context.get('goal_title', '')}\n"
context_section += f"Описание цели: {roadmap_context.get('goal_description', '')}\n"

if roadmap_context.get('tasks'):
    context_section += "\nТекущие задачи:\n"
    for idx, task in enumerate(roadmap_context['tasks'], 1):
        context_section += f"{idx}. {task.get('title', '')}\n"
```

Рисунок 2.11 – Формирование контекста списка задач

Сформированный контекст добавляется к системному своду правил перед отправкой запроса. Благодаря этому система способна не только создавать новые списки задач, но и выполнять редактирование уже существующих.

Важной частью серверной логики является механизм проверки доступа пользователя к списку задач. Перед выполнением операций, система проверяет существование списка задач и принадлежность его текущему пользователю. На рисунке 2.12 представлен фрагмент процедуры проверки доступа.

```
ai_chat_roadmap.py

stmt = select(Roadmap).where(Roadmap.roadmap_id == roadmap_id)
res = await db.execute(stmt)
roadmap = res.scalars().first()

if not roadmap:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="Роудман не найден"
    )

if goal and goal.user_id == user_id:
    return roadmap
```

Рисунок 2.12 – Проверка доступа пользователя к списку задач

В случае отсутствия списка задач или доступа к нему, система возвращает ошибку. Реализация подобного механизма обеспечивает защиту

пользовательских данных.

Для управления содержимым списка реализованы процедуры создания, обновления и удаления задач. При создании новой задачи формируется объект типа Task. После этого объект сохраняется в базе данных. На рисунке 2.13 представлен фрагмент создания задачи.

```
ai_chat_roadmap.py

task = Task(
    roadmap_id=roadmap.roadmap_id,
    title=title,
    description=description,
    order_index=order_index or 0,
    completed=False,
)

db.add(task)
await db.commit()
await db.refresh(task)
```

Рисунок 2.13 – Фрагмент создания задачи

После сохранения изменений выполняется обновление объекта задачи, что позволяет получить актуальные данные из базы данных. Для отслеживания прогресса выполнения списка задач реализована процедура изменения статуса задачи. На рисунке 2.14 представлен фрагмент обновления статуса выполнения.

```
task_service.py

task.completed = bool(completed)
task.completed_at = datetime.utcnow() if completed else None

db.add(task)
await db.commit()
await db.refresh(task)
```

Рисунок 2.14 – Фрагмент обновления статуса выполнения

Использование поля completed_at позволяет отмечать пункты списка как выполненные, тем самым позволяет отслеживать текущий прогресс и контролировать состояние всего списка. Все изменения сразу же фиксируются в

текущей рабочей сессии базы данных. На финальном этапе система безопасно сохраняет обновленные данные и синхронизирует их. Это гарантирует, что в приложении всегда будет отображаться только самая актуальная и корректная информация.

На рисунке 2.15 представлен маршрут для функции создания задачи.

```
roadmap.router.py

@router.post(
   ("/{roadmap_id}/tasks",
    response_model=TaskResponse,
    status_code=status.HTTP_201_CREATED,
)

async def create_task(
    roadmap_id: int,
    task_data: TaskCreate,
    current_user: User = Depends(get_current_user),
):
```

Рисунок 2.15 – Маршрут создания задачи

Взаимодействие клиентской и серверной частей приложения реализовано с использованием REST API интерфейса на базе фреймворка FastAPI. Для каждого типа операции реализован отдельный маршрут, обеспечивающий обработку HTTP-запросов и передачу данных между клиентом и сервером.

В архитектуре взаимодействия применяются стандартные методы протокола HTTP, что позволяет четко разграничить операции чтения, создания, обновления и удаления данных. Обмен информационными пакетами между сторонами осуществляется в универсальном формате JSON, обеспечивающем высокую скорость использования объектов.

Для проверки корректности поступающих от клиентской стороны сведений задействованы встроенные механизмы автоматической валидации, которые отсекают некорректные запросы на этапе их приема.

3 Рекомендации по эксплуатации продукта

3.1 Назначение и условия применения

Веб-приложение декомпозиции целей предназначен для автоматизации процессов планирования, структурирования и контроля выполнения пользовательских целей. Основной функционал системы включает обработку запросов на естественном языке, генерацию последовательности задач с использованием ИИ-ассистента, отображение сформированного плана в удобном формате для просмотра и редактирования, а также сохранение созданных списков задач в базе данных для последующего доступа и использования.

Система может применяться как для личного планирования, так и в образовательной и профессиональной деятельности. Веб-приложение позволяет формировать планы подготовки к обучению, освоению новых навыков, выполнению проектов и достижению долгосрочных целей. Использование ИИ-ассистента позволяет сократить время на ручное составление плана и повысить качество структурирования задач.

Для работы с веб-приложением требуется персональный компьютер, ноутбук или мобильное устройство с доступом к сети Интернет. Поддерживаются операционные системы Windows, macOS, Android и iOS. Корректное функционирование системы обеспечивается при использовании современных веб-браузеров с поддержкой актуальных веб-стандартов, включая Google Chrome, Mozilla Firefox, Microsoft Edge и Safari. Для стабильной работы интерфейса и взаимодействия с ИИ-сервисом рекомендуется интернет-соединение со скоростью не менее 5 Мбит/с.

Работа с системой предполагает наличие базовых навыков взаимодействия с веб-интерфейсами, включая навигацию по страницам, заполнение текстовых форм и использование элементов управления. Эффективность формирования плана зависит от корректности и полноты введенного описания цели.

3.2 Подготовка к работе

Доступ к системе осуществляется через веб-браузер без необходимости установки дополнительного программного обеспечения.

Для начала работы необходимо открыть веб-браузер и перейти по адресу targetl.site. На стартовой странице, представленной на рисунке 3.1 присутствует информация о основном функционале, а также элементы в виде кнопок для перехода к окнам аутентификации.

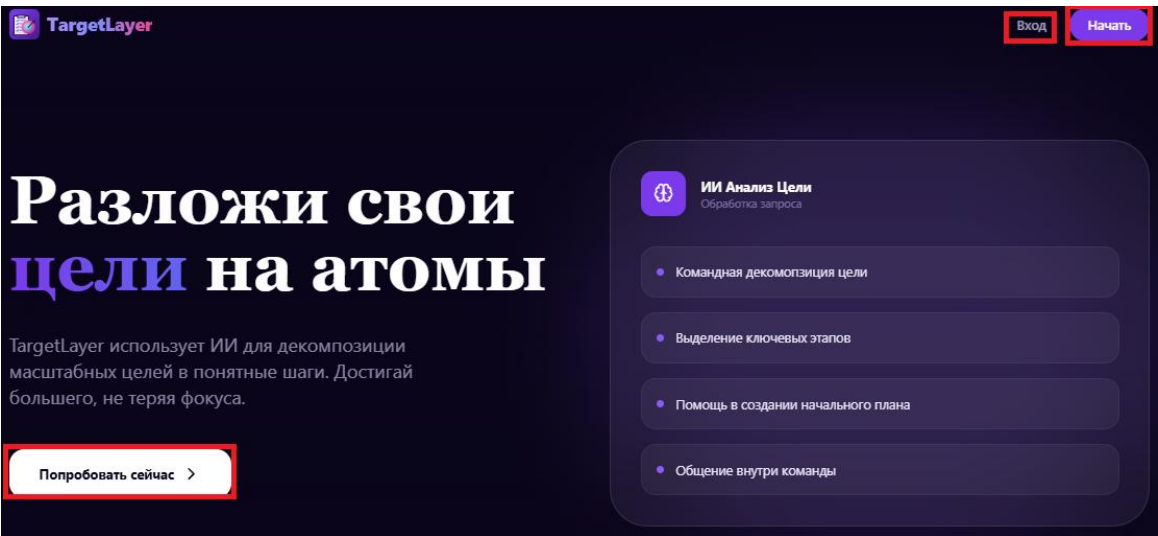


Рисунок 3.1 – Форма регистрации

Красными контурами выделены ключевые навигационные элементы, обеспечивающие переход к процедурам авторизации и регистрации в системе. В правом верхнем углу интерфейса для этой цели предусмотрена кнопка «Начать», расположенная рядом с кнопкой «Вход». Дополнительным интерактивным компонентом выступает контрастная кнопка «Попробовать сейчас», размещенная в левой нижней части экрана под главным информационным блоком и предназначенная для инициализации процесса создания учетной записи.

Активация любого из указанных управляющих элементов перенаправляет к интерфейсам проверки прав доступа. В зависимости от выбранного сценария взаимодействия – первичного создания профиля или входа в уже существующий аккаунт – открываются соответствующие экранные формы, содержащие стандартизированные структурой базы данных, поля для ввода персональных

данных, представленных на рисунке 3.2.

The image shows two overlapping login and registration forms. The background form is titled 'Создать аккаунт' (Create account) with the subtitle 'Начните декомпозировать свои мечты в реальные задачи'. It contains fields for 'Никнейм' (username123), 'Имя' (Александр), 'Фамилия' (Иванов), 'Отчество (опционально)' (Сергеевич), 'Email' (name@example.com), 'Пароль' (masked), and 'Подтвердите пароль' (masked). A purple button 'Создать аккаунт' is at the bottom, with a link 'Уже есть аккаунт? Войти' below it. The foreground form is titled 'С возвращением' (Welcome back) with the subtitle 'Войдите, чтобы продолжить работу над вашими целями'. It contains fields for 'Email' (name@example.com) and 'Пароль' (masked), with a link 'Забыли пароль?' next to the password field. A purple button 'Войти в аккаунт' is at the bottom, with a link 'Нет аккаунта? Зарегистрироваться' below it.

Рисунок 3.2 – Формы аутентификации

После успешного прохождения процедур регистрации и авторизации открывается доступ к основному функционалу системы. Навигация по ключевым разделам приложения осуществляется посредством вертикального бокового меню, стационарно расположенного в левой части интерфейса для обеспечения быстрого переключения между экранами. В структуру данного навигационного блока интегрированы следующие разделы:

- профиля;
- команды;
- поиска пользователей;
- сообщений;
- ИИ-помощника;
- списков задач (роудмапов).

Для первичного формирования структуры целей необходимо перейти в раздел «ИИ-помощник», графический интерфейс которого представлен на

рисунке 3.2, и инициализировать новый диалог.

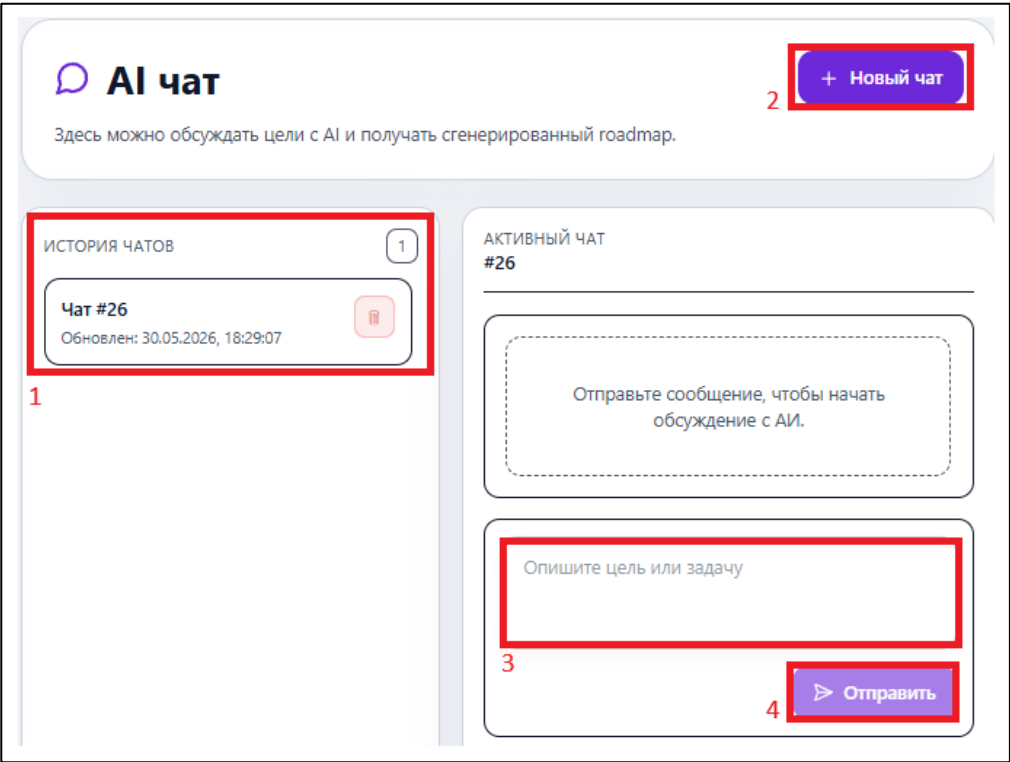


Рисунок 3.2 – Интерфейс для общения с ИИ-помощником

Интерфейс ИИ-помощника реализован в виде диалогового окна и включает следующие элементы:

- «1» – панель «История чатов», расположенная в левой части экрана. Данный элемент предназначен для отображения списка ранее созданных диалогов. Внутри карточки каждого чата предусмотрена кнопка со значком корзины для удаления неактуальных сессий из памяти системы;
- «2» – управляющая кнопка «Новый чат», размещенная в правом верхнем углу интерфейса. Она выполняет функцию инициализации новой пустой сессии взаимодействия с интеллектуальным ассистентом.
- «3» – текстовое поле ввода. Элемент находится в нижней части рабочей зоны активного чата и служит для ручного ввода подробного описания требуемой цели,
- «4» – функциональная кнопка «Отправить», расположенная непосредственно под полем ввода. Данный компонент служит для подтверждения

и отправки сформированного текстового запроса на серверную часть для последующей обработки ИИ-сервисом.

После отправки запроса система осуществляет обработку переданного текста и формирует структурированный ответ в рамках активного диалога. Пример успешного взаимодействия с ИИ-помощником и демонстрация сгенерированного результата представлены на рисунке 3.3.

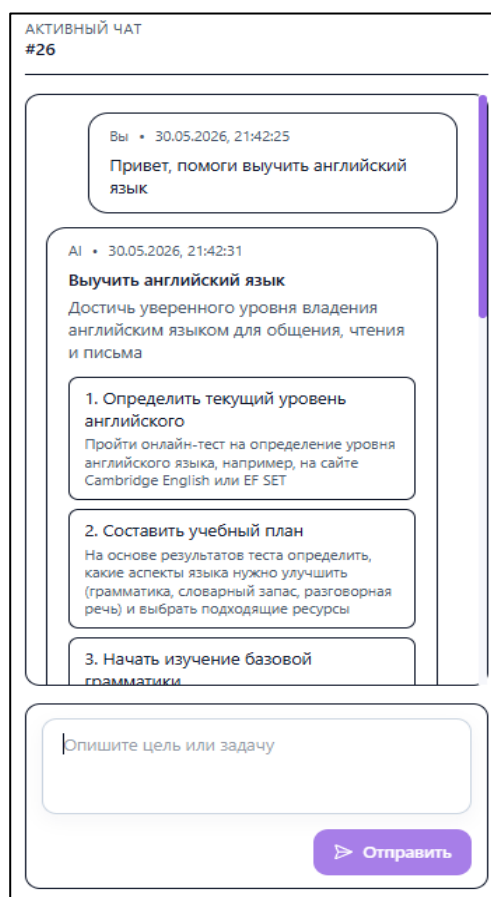


Рисунок 3.3 – Успешный результат генерации списка задач

В ответном сообщении от ИИ-ассистента выводится автоматически сформированный план, заголовок которого соответствует целевому запросу. Ниже генерируется декомпозированный список, где каждый этап представлен в виде отдельной пронумерованной карточки с четким названием шага и его лаконичным описанием. Структурированный таким образом ответ позволяет визуально оценить последовательность действий, необходимых для достижения поставленного результата, непосредственно внутри интерфейса чата.

Сформированный ИИ-ассистентом план автоматически интегрируется в общую систему планирования.

Для работы с готовыми планами, отслеживания их выполнения и ручного редактирования предусмотрен специализированный экран управления, представленный на рисунке 3.4.

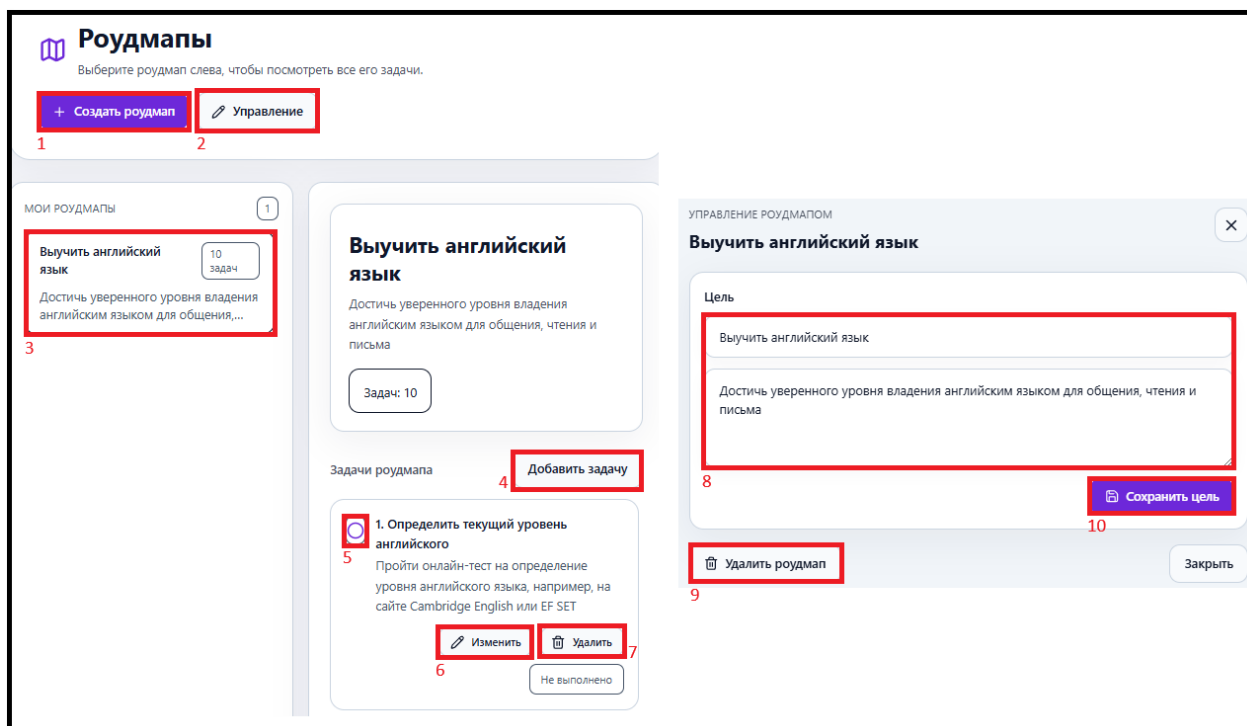


Рисунок 3.2 – Интерфейс управления списками задач

На данном рисунке выделены и пронумерованы ключевые интерактивные элементы управления списком задач:

- «1» – управляющая кнопка «Создать роудмап», служащая для ручного добавления нового списка задач, ручным способом;
- «2» – кнопка «Управление», активирующая панель редактирования параметров текущего списка задач;
- «3» – боковая панель, где отображаются карточки всех списков у пользователя указанием общего количества входящих в него задач;
- «4» – интерактивный элемент «Добавить задачу», позволяющий вручную расширить текущий список этапов;

– «5» – графический маркер статуса выполнения, предназначенный для быстрой фиксации завершения конкретного шага;

– «6» и «7» – функциональные кнопки «Изменить» и «Удалить», интегрированные в карточку задачи для редактирования или очистки конкретного пункта;

– «8» – текстовые поля ввода «Цель», расположенные на панели управления и предназначенные для корректировки названия и описания текущего направления;

– «9» – кнопка «Удалить роудмап», обеспечивающая безвозвратное удаление списка из системы;

– «10» – кнопка действия «Сохранить цель», фиксирующая все внесенные в текстовые поля изменения в рамках текущей сессии.

Таким образом, представленный интерфейс обеспечивает полный цикл оперирования сгенерированными планами, позволяя гибко модифицировать структуру задач в соответствии с текущими потребностями.

Пользователь получает возможность полностью адаптировать предложенный план под индивидуальные особенности своего рабочего или учебного процесса. Пошаговое выполнение задач сопровождается наглядным изменением общего прогресса, что помогает контролировать остаток запланированных работ. При возникновении новых обстоятельств можно расширять списки, не нарушая при этом общую логику уже созданного списка.

Важно учитывать, что автоматически сгенерированный ИИ-помощником план не является абсолютно точным планом действий. Полученный результат представляет собой исключительно базовую отправную точку, концептуальный ориентир для начала работы над задачей. Пользователю необходимо самостоятельно проанализировать предложенные этапы. Окончательная адаптация, детализация и корректировка шагов в соответствии со спецификой конкретного проекта остаются целиком на усмотрение пользователя.

4 Тестирование программного продукта

4.1 Выбор стратегии тестирования

Для комплексной проверки работоспособности веб-приложения выбрана стратегия, сочетающая функциональное тестирование методом черного ящика и модульное тестирование компонентов серверной части. Такая комбинация позволяет одновременно оценить систему с позиции конечного пользователя и проверить стабильность внутренней логики обработки данных.

Функциональный метод ориентирован на верификацию программного продукта без анализа структуры исходного кода путем выполнения пользовательских действий и сверки полученного результата с требованиями проекта. В рамках этого этапа проверялись ключевые сценарии, включающие регистрацию и авторизацию учетных записей, генерацию декомпозированных списков задач через ИИ-сервис, создание рабочих команд, обмен сообщениями во встроенном чате, а также корректность и стабильность отображения элементов интерфейса.

Для верификации изолированных программных модулей бэкенда параллельно применялось модульное тестирование серверных компонентов. Основное внимание здесь уделялось проверке правильности работы реализованных API-маршрутов фреймворка FastAPI, обработке входящих HTTP-запросов и ответов сервера, механизмам автоматической валидации передаваемых данных и стабильности выполнения операций при взаимодействии с базой данных PostgreSQL.

Реализация данной комплексной стратегии направлена на подтверждение надежности интеграции между клиентской и серверной частями веб-приложения, а также на обеспечение безопасности обработки пользовательских данных. Ожидается, что проведение запланированных тестов позволит своевременно выявить и устранить возможные критические ошибки на стыке различных компонентов программного продукта.

4.2 Разработка методов тестирования

Для автоматизированной проверки отдельных компонентов системы разработаны модульные тесты на языке Python с использованием библиотеки unittest. Применение асинхронных методов позволило протестировать бизнес-логику FastAPI-приложения, его взаимодействие с базой данных PostgreSQL и обработку запросов в условиях, близких к реальной эксплуатации.

Тестирование охватило ключевые функции веб-приложения: регистрацию и авторизацию учетных записей, генерацию списков задач ИИ-сервисом, создание команд и обмен сообщениями в чате. Особое внимание уделялось механизму аутентификации, регулирующему доступ к персональным данным.

В рамках проверки процедуры создания профиля реализовано тестирование, в котором вызывается функцию регистрации с передачей пользовательских параметров, а затем обращается к базе данных для верификации сохраненной информации. В ходе выполнения теста анализируется факт создания записи, точность записи имени и адреса электронной почты. Пример реализации данной проверки представлен на рисунке 4.1.

```
test_register.py

async def test_login_with_existing_user(self):
    username = "test_user"
    email = "test@example.com"
    password = "Secret123!"

    created_user_id = await self.service.register_email(
        username,
        email,
        password,
        "Aleksandr",
        "Aleksandrov",
    )

    result = await self.db.execute(
        select(User).where(User.user_id == created_user_id)
    )
    user = result.scalars().first()

    self.assertIsNotNone(user)
    self.assertEqual(user.email, email)

    authenticated_user_id = await self.service.authenticate_email(
        email,
        password
    )

    self.assertEqual(authenticated_user_id, created_user_id)
```

Рисунок 4.1 – Реализация тестирования функции регистрации

Проверка механизма регистрации дополняется тестированием функции авторизации, поскольку данные процессы являются взаимосвязанными элементами системы аутентификации. В процессе выполнения тестирования предварительно создается учетная запись пользователя, после чего выполняется попытка входа с использованием валидных параметров.

В рамках проверки анализируется точность поиска пользователя в базе данных, сверка введенного пароля и возврат идентификатора авторизованного лица. Реализация проверки функции авторизации представлена на рисунке 4.2.

```
test_auth.py

async def test_register_email_creates_user(self):
    user_id = await self.service.register_email(
        "test_user",
        "test@example.com",
        "Secret123!",
        "Aleksandr",
        "Aleksandrov",
    )

    result = await self.db.execute(
        select(User).where(User.user_id == user_id)
    )
    user = result.scalars().first()

    self.assertIsNotNone(user)
    self.assertEqual(user.username, "test_user")
    self.assertEqual(user.email, "test@example.com")
    self.assertTrue(user.password_hash)

async def test_register_email_requires_username(self):
    with self.assertRaisesRegex(
        ValueError,
        "Имя пользователя обязательно"
    ):
        await self.service.register_email(
            "",
            "test@example.com",
            "Secret123!",
            "Aleksandr",
            "Aleksandrov",
        )
```

Рисунок 4.2 – Реализация тестирования функции авторизации

После проверки механизмов аутентификации выполнялось тестирование функции генерации списков задач при помощи ИИ-сервиса, являющейся основным функциональным элементом веб-приложения.

Для проверки работы данного механизма использовался объект, имитирующий ответ нейросетевой модели, что позволило протестировать обработку данных без необходимости обращения к реальному внешнему API.

В ходе выполнения теста осуществляется отправка пользовательского запроса, формирование списка и фиксация созданных задач в базе данных. Реализация проверки функции генерации списка задач представлена на рисунке 4.3.

```
test_create_roadmap.py

async def test_generate_roadmap_and_save_to_db(self):
    user_id = await self.auth_service.register_email(
        "test_user",
        "test@example.com",
        "Secret123!",
        "Aleksandr",
        "Aleksandrov",
    )
    result = await self.db.execute(
        select(User).where(User.user_id == user_id)
    )
    current_user = result.scalars().first()

    ai_response = {
        "goal_title": "Изучить Python",
        "tasks": [
            {
                "title": "Изучить основы Python",
                "description": "Синтаксис и базовые конструкции",
                "order_index": 0,
            },
        ],
    }

    with patch(
        "app.api.v1.ai.ai_router.ai_service.chat",
        new=AsyncMock(return_value=ai_response),
    ):
        response = await ai_chat(
            AIRoadmapRequest(prompt="Изучить Python"),
            current_user=current_user,
            db=self.db,
        )

        self.assertEqual(response.goal_title, "Изучить Python")
        self.assertEqual(len(response.tasks), 1)

        roadmap_result = await self.db.execute(select(Roadmap))
        roadmap = roadmap_result.scalars().first()

        self.assertIsNotNone(roadmap)
        self.assertEqual(
            roadmap.tasks[0].title,
            "Изучить основы Python"
        )
    )
```

Рисунок 4.3 – Реализация тестирования функции генерации списка задач

Работа с задачами внутри системы предполагает возможность совместного взаимодействия, в связи с чем реализовано тестирование механизма создания команд. В процессе выполнения теста производится создание нового пользователя и вызов функции формирования рабочей группы с последующей проверкой сохранения информации в базе данных. Проверка данного механизма позволяет убедиться в стабильности работы командного функционала системы.

Изм.	Лист	№ докум	Подпись	Дата

НАТКиГ.101700.010.000ПЗ

Лист

47

Реализация проверки функции создания команды представлена на рисунке 4.4.

```
test_create_team.py

async def test_create_team(self):
    user_id = await self.auth_service.register_email(
        "test_user",
        "test@example.com",
        "Secret123!",
        "Александр",
        "Иванов",
    )
    result = await self.db.execute(
        select(User).where(User.user_id == user_id)
    )
    user = result.scalars().first()

    team = await create_team(
        self.db,
        user,
        "Python Team"
    )
    team_result = await self.db.execute(
        select(Team).where(
            Team.team_id == team.team_id
        )
    )
    created_team = team_result.scalars().first()

    self.assertIsNotNone(created_team)
    self.assertEqual(
        created_team.name,
        "Python Team"
    )
```

Рисунок 4.4 – Реализация тестирования функции создания команды

Функционал командного взаимодействия дополняется встроенной системой обмена сообщениями, обеспечивающей коммуникацию между участниками команды в режиме реального времени. Для проверки данного механизма реализован тест отправки сообщений, включающий создание пользователя, команды и командного чата с последующей отправкой текстового сообщения.

После выполнения операции осуществляется обращение к базе данных для проверки сохранения текста сообщения, идентификатора пользователя и идентификатора чата. Проверка данного функционала позволяет убедиться в корректности работы механизма обмена сообщениями и взаимодействия пользователей внутри системы.

Реализация проверки функции отправки сообщений представлена на рисунке 4.5.


```

test_create_message.py

async def test_create_team_message(self):
    user_id = await self.auth_service.register_email(
        "test_user",
        "test@example.com",
        "Secret123!",
        "Александр",
        "Иванов",
    )
    result = await self.db.execute(
        select(User).where(User.user_id == user_id)
    )
    user = result.scalars().first()

    team = await create_team(
        self.db,
        user,
        "Python Team"
    )
    chat = await get_or_create_team_chat(
        self.db,
        team_id=team.team_id,
        user_id=user.user_id,
    )
    message = await send_chat_message(
        self.db,
        chat_id=chat.chat_id,
        user_id=user.user_id,
        content="Привет, команда",
    )
    message_result = await self.db.execute(
        select(Message).where(
            Message.message_id == message.message_id
        )
    )
    created_message = message_result.scalars().first()

    self.assertIsNotNone(created_message)
    self.assertEqual(
        created_message.content,
        "Привет, команда"
    )

```

Рисунок 4.5 – Реализация тестирования функции отправки сообщения

Для комплексного подтверждения работоспособности разработанного веб-приложения сформированы протоколы тестирования, отражающие результаты параллельной проверки пользовательского интерфейса и серверной части. Проверка базовых компонентов разграничения доступа выполнялась в первую очередь, так как эти функции обеспечивают вход в защищенные разделы платформы. Сопоставление результатов пользовательского и модульного тестирования для процедур регистрации приведено ниже в таблицах 4.1 и 4.2.

Таблица 4.1 – Протокол модульного тестирования авторизации

Наименование	Описание
1	2
Test Case #	1
Приоритет тестирования	Высокий
Название тестирования	Авторизация пользователя

Продолжение таблицы 4.1

1	2
Цель тестирования	Проверка работы функции регистрации и последующей авторизацией пользователя
Шаги тестирования	1 Создание тестового пользователя в базе данных 2 Проверка сохранения учетных данных 3 Выполнение авторизации пользователя по электронной почте и паролю 4 Сравнение идентификатора авторизованного пользователя с идентификатором созданного пользователя
Данные тестирования	1 Имя пользователя – «username» 2 Электронная почта – «login@example.com» 3 Пароль – «Secret123!»
Ожидаемый результат	Пользователь успешно создается в базе данных. Авторизация выполняется штатно. Возвращаемый идентификатор пользователя соответствует созданной учетной записи
Фактический результат	Тест выполнен успешно. Пользователь создан в базе данных, авторизация выполнена без ошибок, идентификаторы совпадают

Таблица 4.3 – Протокол пользовательского тестирования авторизации

Наименование	Описание
Test Case #	2
Приоритет тестирования	Высокий
Название тестирования	Авторизация пользователя
Шаги тестирования	1 Открытие страницы «Вход в систему» 2 Ввод электронной почты и пароля 3 Нажатие кнопки «Войти»
Данные тестирования	1 Электронная почта – «alex@gmail.com» 2 Пароль – «Secret123! »
Ожидаемый результат	Выполняется успешная авторизация пользователя и загрузка главной страницы
Фактический результат	Авторизация выполнена успешно. Главная страница приложения отображается корректно

Анализ полученных данных подтвердил надежность механизма регистрации как со стороны графического интерфейса, так и на уровне серверных алгоритмов. Выполненные проверки зафиксировали создание учетной записи, точное сохранение пользовательских параметров и стабильную работу механизма автоматического входа. Функционирование системы аутентификации дополнительно проверено при изолированном тестировании процесса входа.

Модульные тесты подтвердили точность поиска записей в базе данных, валидацию пароля и возврат токена авторизованного пользователя.

Результаты этих проверок зафиксированы в таблицах 4.3 и 4.4.

Таблица 4.4 – Протокол модульного тестирования регистрации

Наименование	Описание
Test Case #	3
Приоритет тестирования	Высокий
Название тестирования	регистрации пользователя
Цель тестирования	Проверка создания учетной записи пользователя и валидации обязательных данных
Шаги тестирования	1 Генерация уникальных пользовательских данных 2 Вызов функции регистрации пользователя 3 Проверка сохранения данных пользователя в базе данных 4 Проверка сохранения имени пользователя, электронной почты и хэша пароля 5 Проверка обработки ошибки при отсутствии имени пользователя
Данные тестирования	1 Имя пользователя – «test_user» 2 Электронная почта – «test_user@example.com» 3 Пароль – «Secret123!» 4 Имя – «Aleksandr» 5 Фамилия – «Aleksandrov»
Ожидаемый результат	Пользователь успешно создается и сохраняется в базе данных. Все поля учетной записи сохраняются корректно. При передаче пустого имени пользователя система генерирует ошибку валидации
Фактический результат	Тест выполнен успешно. Пользователь корректно создан и сохранен в базе данных, данные учетной записи соответствуют переданным значениям, ошибка валидации успешно обработана

Таблица 4.2 – Протокол пользовательского тестирования регистрации

Наименование	Описание
Test Case #	4
Приоритет тестирования	Высокий
Название тестирования	Регистрация пользователя
Шаги тестирования	1 Открытие страницы «Регистрация» 2 Ввод данных пользователя 3 Нажатие кнопки «Зарегистрироваться»
Данные тестирования	1 Имя пользователя – «test_user» 2 Электронная почта – «alex@gmail.com» 3 Пароль – «Secret123!»
Ожидаемый результат	Учетная запись успешно создается. Выполняется переход в систему
Фактический результат	Регистрация выполнена успешно. Пользователь авторизован в системе

Проведенное тестирование подтвердило отсутствие сбоев в логике входа в систему и стабильное взаимодействие клиентской и серверной частей.

Верификация ключевого интеллектуального компонента, отвечающего за декомпозицию целей. Проверка пользовательского сценария сфокусирована на обработке текстового запроса интерфейсом ИИ-помощника и выводе сгенерированного плана.

Модульное тестирование подтверждало внутреннюю структуру передаваемых пакетов и фиксацию дорожных карт в реляционной базе данных. Полученные результаты сведены в таблицы 4.5 и 4.6.

Таблица 4.5 – Протокол модульного тестирования генерации списка задач

Наименование	Описание
Test Case #	5
Приоритет тестирования	Высокий
Название тестирования	Генерация списка задач
Цель тестирования	Проверка обработки ответа ИИ-сервиса, формирования плана и сохранения задач в базе данных
Шаги тестирования	1 Создание тестового пользователя 2 Формирование тестового запроса на генерацию плана 3 Имитация ответа ИИ-сервиса при помощи объекта AsyncMock 4 Вызов функции генерации списка задач 5 Проверка сохранения плана и задач в базе данных 6 Проверка структуры и содержимого сформированных задач
Данные тестирования	1 Запрос пользователя – «Сделай короткий план изучения Python на 2 недели» 2 Название цели – «Изучить Python за 2 недели» 3 Описание задачи – «Освоить основы языка»
Ожидаемый результат	Система корректно обрабатывает ответ ИИ-сервиса, создает план, сохраняет список задач в базе данных и возвращает корректную структуру данных
Фактический результат	Тест выполнен успешно. план и задачи созданы и сохранены в базе данных, структура ответа соответствует ожидаемому результату

Таблица 4.6 – Протокол пользовательского тестирования генерации списка задач

Наименование	Описание
1	2
Test Case #	6
Приоритет тестирования	Высокий
Название тестирования	Генерация списка задач

Продолжение таблицы 4.6

1	2
Шаги тестирования	1 Переход в раздел «ИИ-помощник» 2 Ввод описания цели 3 Нажатие кнопки «Отправить»
Данные тестирования	1 Цель – «Изучить Python за 2 месяца, уделяя 5 часов в неделю»
Ожидаемый результат	Система генерирует структурированный список задач и отображает его на экране
Фактический результат	Список задач успешно сформирован и сохранен в системе

Результаты тестирования подтвердили работоспособность механизма генерации задач, успешную обработку пользовательских запросов и стабильность формирования списка задач. Проверка серверной части подтвердила работоспособность сохранения сформированных задач и структуры списков задач в базе данных приложения.

В рамках пользовательского тестирования анализировалась корректность взаимодействия с интерфейсом создания команды, отображение созданной команды в системе и работа командного чата. Модульные тесты позволили подтвердить стабильность работы серверных функций, отвечающих за создание команд, взаимодействие с чатами и сохранение сообщений в базе данных.

Полученные результаты пользовательского и модульного тестирования функции создания команды представлены в таблицах 4.7 и 4.8.

Таблица 4.7 – Протокол модульного тестирования создание команды

Наименование	Описание
1	2
Test Case #	7
Приоритет тестирования	Высокий
Название тестирования	Модульное тестирование функции создания команды
Цель тестирования	Проверка корректности создания команды и сохранения информации о команде в базе данных
Шаги тестирования	1 Создание тестового пользователя 2 Выполнение функции создания команды 3 Сохранение команды в базе данных 4 Получение информации о созданной команде из базы данных 5 Проверка корректности названия команды

Продолжение таблицы 4.7

Данные тестирования	1 Имя пользователя – «team_user» 2 Электронная почта – «team_user_@example.com» 3 Название команды – «Команда»
Ожидаемый результат	Команда успешно создается и сохраняется в базе данных. Название команды соответствует переданному значению
Фактический результат	Тест выполнен успешно. Команда корректно создана и сохранена в базе данных, название команды соответствует ожидаемому значению

Таблица 4.8 – Протокол пользовательского тестирования создание команды

Наименование	Описание
Test Case #	8
Приоритет тестирования	Высокий
Название тестирования	Создание команды
Шаги тестирования	1 Переход в раздел «Команды» 2 Нажатие кнопки «Создать команду» 3 Ввод названия команды 4 Нажатие кнопки «Создать»
Данные тестирования	1 Название команды – «Team_test»
Ожидаемый результат	Команда успешно создается и отображается в списке команд
Фактический результат	Команда успешно создана и доступна пользователю

Проведенные проверки подтвердили стабильную работу механизма создания команд, сохранения информации о команде в базе данных и успешную привязку команды к пользователю. Работа интерфейса командного взаимодействия также соответствовала ожидаемым результатам тестирования.

Проверка функционала обмена сообщениями выполнялась в рамках тестирования командного взаимодействия пользователей внутри системы. В процессе пользовательского тестирования анализировалась соответствие проектируемых норм отображения сообщений в командном чате, обработка отправки текстовых сообщений и стабильность работы интерфейса в режиме взаимодействия пользователей. Модульное тестирование позволяло проверить работоспособность создания сообщений, сохранения текста сообщения, идентификаторов пользователя и чата, а также взаимодействия компонентов системы обмена сообщениями с базой данных.

Полученные результаты пользовательского и модульного тестирования функции создания командного сообщения представлены в таблицах 4.9 и 4.10.

Таблица 4.10 – Протокол модульного тестирования создания сообщения

Наименование	Описание
Test Case #	9
Приоритет тестирования	Средний
Название тестирования	Отправка сообщений в командный чат
Цель тестирования	Проверка корректности создания сообщения, взаимодействия с командным чатом и сохранения данных сообщения в базе данных
Шаги тестирования	1 Создание тестового пользователя 2 Создание команды 3 Создание или получение командного чата 4 Отправка текстового сообщения в чат 5 Проверка сохранения сообщения в базе данных 6 Проверка корректности текста сообщения, идентификатора пользователя и идентификатора чата
Данные тестирования	1 Название команды – «Команда» 2 Текст сообщения – «Привет, команда» 3 Электронная почта пользователя – «mes@example.com»
Ожидаемый результат	Сообщение успешно создается и сохраняется в базе данных. Текст сообщения, идентификатор пользователя и идентификатор чата сохраняются корректно
Фактический результат	Тест выполнен успешно. Сообщение корректно создано и сохранено в базе данных, данные сообщения соответствуют ожидаемым значениям

Таблица 4.9 – Протокол пользовательского тестирования создания сообщения

Наименование	Описание
Test Case #	10
Приоритет тестирования	Средний
Название тестирования	Отправка сообщения в командный чат
Шаги тестирования	1 Переход в раздел командного чата 2 Ввод текстового сообщения 3 Нажатие кнопки «Отправить»
Данные тестирования	1 Сообщение – «Привет, команда!»
Ожидаемый результат	Сообщение успешно отправляется и отображается в чате
Фактический результат	Сообщение корректно отображается в командном чате

Результаты тестирования подтвердили стабильную работу механизма обмена сообщениями, соответствие проектируемым нормам отображения сообщений внутри командного чата и успешное сохранение данных сообщений в базе данных приложения.

4.3 Протоколы тестирования

По результатам выполненных проверок сформированы сводные протоколы, фиксирующие итоги верификации каждого эксплуатационного сценария. Данные документы включают идентификатор проверки, наименование процедуры, итоговый статус и краткое аналитическое заключение о соответствии фактического поведения программного продукта запланированным требованиям.

Сводные результаты сквозного тестирования пользовательского интерфейса методом черного ящика представлены в таблице 4.10.

Таблица 4.10 – Сводный протокол пользовательского тестирования

ID теста	Название теста	Статус	Комментарий
Test Case # 2	Авторизация пользователя	Пройден	Проверка учетных данных и загрузка главного экрана приложения осуществляются штатно
Test Case # 4	Регистрация пользователя	Пройден	Учетная запись фиксируется в системе, перенаправление на форму входа выполняется без задержек
Test Case # 6	Генерация списка задач	Пройден	Текстовый запрос обрабатывается успешно, план выводится в диалоговом окне
Test Case # 8	Создание команды	Пройден	Создание команд и отображение их в боковой панели навигации реализовано в полном объеме
Test Case # 10	Отправка сообщения в командный чат	Пройден	Текст сообщения мгновенно появляется в общем чате выбранной команды

Сводные результаты автоматизированной проверки изолированных модулей серверной части приложения, подтверждающие точность работы бизнес-логики бэкенда и алгоритмов интеграции с реляционной базой данных, приведены в таблице 4.11.

Таблица 4.11 – Сводный протокол модульного тестирования

ID теста	Название теста	Статус	Комментарий
Test Case # 1	test_auth	Пройден	Сессия пользователя создается в соответствии с введенными данными
Test Case # 3	test_register	Пройден	Подтверждена точность записи параметров пользователя в базу данных PostgreSQL и генерация безопасного хэша пароля

Продолжение таблицы 4.11

Test Case # 5	test_create_roadmap	Пройден	Верифицирована структура входящих пакетов и сохранение сгенерированных карточек задач в рамках активной сессии
Test Case # 7	test_create_team	Пройден	Команда успешно создается и фиксируется в базе данных
Test Case # 8	test_create_message	Пройден	Подтверждена фиксация текста, связей авторов и уникальных номеров чатов на уровне серверных маршрутов FastAPI

Анализ результатов проведённого тестирования подтверждает функционирование разработанного веб-приложения и выполнение предусмотренных пользовательских сценариев. Выполненные проверки позволили оценить качество реализации основных компонентов системы, включая механизмы взаимодействия между клиентской и серверной частями, обработку пользовательских действий и управление данными.

Результаты тестирования показали стабильность работы реализованных функций, отсутствие критических ошибок при выполнении основных операций и согласованность работы отдельных компонентов приложения. Проверка различных сценариев использования подтвердила возможность полноценного применения системы для решения поставленных задач.

В ходе испытаний также подтверждена работоспособность обработки пользовательских данных, выполнения операций по созданию и изменению информации, а также своевременного отображения результатов работы системы в пользовательском интерфейсе. Реализованные механизмы обеспечивают предсказуемое поведение приложения при выполнении типовых действий пользователей и поддерживают целостность данных в процессе эксплуатации.

Полученные результаты свидетельствуют о том, что разработанное приложение соответствует поставленным целям проекта и может использоваться в качестве инструмента для декомпозиции целей, планирования задач и организации процесса их достижения.

Заключение

В ходе выполнения дипломного проекта достигнута поставленная цель – разработано веб-приложение для декомпозиции целей, позволяющее организовать процесс планирования и контроля их достижения. Созданная система предоставляет пользователям возможность структурировать поставленные задачи, отслеживать прогресс их выполнения и эффективно организовывать как индивидуальную, так и командную деятельность.

Разработка приложения основывалась на анализе существующих систем управления задачами и особенностей организации процесса целеполагания. Проведённое исследование позволило определить основные требования к программному продукту и выбрать технологические решения, обеспечивающие удобство использования, надёжность работы и возможность дальнейшего расширения функциональности.

В результате реализована система, объединяющая инструменты управления целями и задачами, средства командного взаимодействия, обмена сообщениями и интеллектуальной поддержки пользователей. Использование клиент-серверной архитектуры позволило обеспечить согласованную работу всех компонентов приложения, а также организовать централизованное хранение и обработку данных. При разработке особое внимание уделялось построению гибкой архитектуры, способной поддерживать дальнейшее развитие проекта без существенных изменений существующей структуры.

Практическая реализация поставленных задач позволила создать удобный инструмент для организации деятельности и контроля достижения целей. Проведённое функциональное, модульное и пользовательское тестирование подтвердило корректность работы реализованного функционала и готовность системы к эксплуатации.

Таким образом, поставленные в рамках дипломного проекта задачи успешно решены, а разработанное веб-приложение соответствует поставленным требованиям и обеспечивает реализацию предусмотренного функционала.

Библиография

Нормативно-правовые акты:

1 ГОСТ Р 2.105-2019 ЕСКД. Общие требования к текстовым документам. – Москва: Стандартинформ, 2019. – 36 страниц (дата обращения: 22.05.2026).

2 ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва: Стандартинформ, 2018. – 128 страниц (дата обращения: 22.05.2026).

3 ГОСТ 15.016-2016. Техническое задание. – Москва: Стандартинформ, 2017. – 30 страниц (дата обращения: 22.05.2026).

4 ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения – Москва: Стандартинформ, 1990. – 60 страниц (дата обращения: 22.05.2026).

Электронные ресурсы:

1 Руководство по работе React. – URL: <https://react.dev/> – Текст: электронный (дата обращения: 22.05.2026).

4 Руководство PostgreSQL. – URL: <https://www.postgresql.org/docs/> – Текст: электронный (дата обращения: 22.05.2026).

8 Руководство Docker. – URL: <https://docs.docker.com/> – Текст: электронный (дата обращения: 22.05.2026).

9 NGINX документация. – URL: <https://nginx.org/en/docs/> – Текст: электронный (дата обращения: 22.05.2026).

10 JWT Introduction and Documentation. – URL: <https://jwt.io/introduction> – Текст: электронный (дата обращения: 22.05.2026).

11 Tailwind CSS Документация. – URL: <https://tailwindcss.com/docs> – Текст: электронный (дата обращения: 22.05.2026).

14 Swagger Документация. – URL: <https://swagger.io/docs/> – Текст: электронный (дата обращения: 22.05.2026).

Приложение А

(Обязательное)

ER-диаграмма

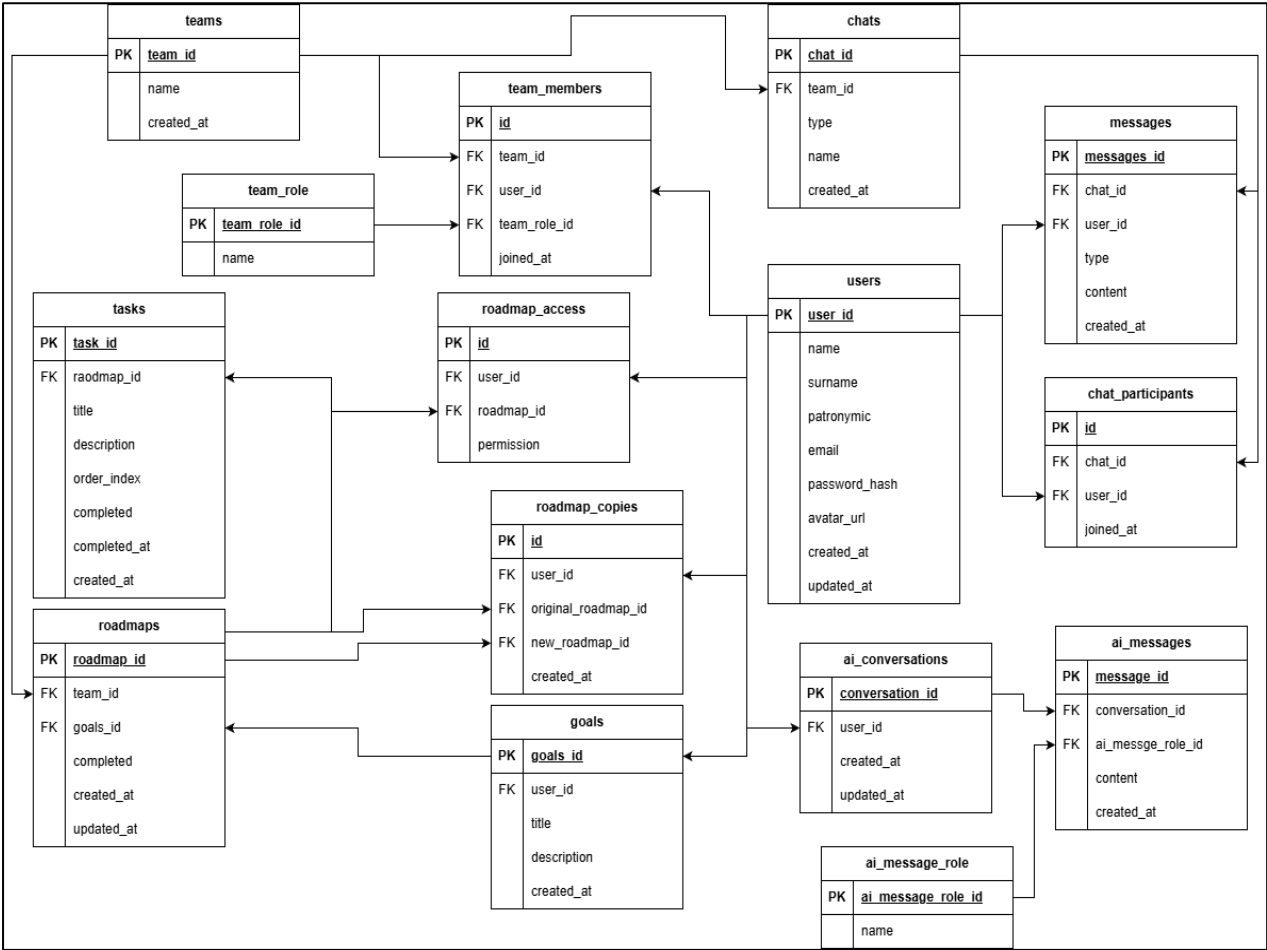


Рисунок А.1 – ER-диаграмма

Приложение Б

(Обязательное)

Словарь данных

Таблица Б.1 – Сущность Users

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор пользователя
email	varchar	No	Адрес электронной почты
password_hash	varchar	No	Хэш пароля
name	varchar	No	Имя пользователя
avatar_url	text	Yes	Ссылка на изображение профиля
created_at	timestamp	No	Дата регистрации
updated_at	timestamp	No	Дата изменения записи

Таблица Б.2 – Сущность Goals

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор цели
user_id	uuid	No	Ссылка на пользователя
title	varchar	No	Наименование цели
description	text	Yes	Описание цели
status	varchar	No	Статус выполнения
created_at	timestamp	No	Дата создания
updated_at	timestamp	No	Дата изменения

Таблица Б.3 – Сущность Roadmaps

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор плана
goal_id	uuid	No	Ссылка на цель
team_id	uuid	Yes	Ссылка на команду
title	varchar	No	Наименование плана
created_at	timestamp	No	Дата создания
updated_at	timestamp	No	Дата изменения

Таблица Б.4 – Сущность Tasks

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор задачи
roadmap_id	uuid	No	Ссылка на план
title	varchar	No	Наименование задачи
description	text	Yes	Описание задачи
status	varchar	No	Статус выполнения
due_date	timestamp	Yes	Срок выполнения
created_at	timestamp	No	Дата создания

Таблица Б.5 – Сущность Teams

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор команды
name	varchar	No	Наименование команды
description	text	Yes	Описание команды
created_at	timestamp	No	Дата создания
updated_at	timestamp	No	Дата изменения

Таблица Б.6 – Сущность Team role

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор роли
name	varchar	No	Наименование роли
description	text	Yes	Описание роли

Таблица Б.7 – Сущность Team_members

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор записи
team_id	uuid	No	Ссылка на команду
user_id	uuid	No	Ссылка на пользователя
role_id	uuid	No	Ссылка на роль участника
joined_at	timestamp	No	Дата вступления в команду

Таблица Б.8 – Сущность Chats

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор чата
team_id	uuid	No	Ссылка на команду
name	varchar	No	Наименование чата
created_at	timestamp	No	Дата создания

Таблица Б.9 – Сущность Messages

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор сообщения
chat_id	uuid	No	Ссылка на чат
user_id	uuid	No	Ссылка на автора
content	text	No	Текст сообщения
created_at	timestamp	No	Дата отправки

Таблица Б.10 – Сущность Chat participants

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор записи
chat_id	uuid	No	Ссылка на чат
user_id	uuid	No	Ссылка на пользователя
joined_at	timestamp	No	Дата подключения к чату

Таблица Б.11 – Сущность Roadmap access

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор доступа
roadmap_id	uuid	No	Ссылка на план
user_id	uuid	No	Ссылка на пользователя
access_type	varchar	No	Тип доступа
created_at	timestamp	No	Дата предоставления

Таблица Б.12 – Сущность Roadmap copies

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор копии
roadmap_id	uuid	No	Ссылка на исходный план
user_id	uuid	No	Владелец копии
created_at	timestamp	No	Дата создания копии

Таблица Б.13 – Сущность AI conversations

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор диалога
user_id	uuid	No	Ссылка на пользователя
title	varchar	Yes	Название диалога
created_at	timestamp	No	Дата создания

Таблица Б.14 – Сущность AI_message_role

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор роли
name	varchar	No	Наименование роли
description	text	Yes	Описание роли

Таблица Б.15 – Сущность AI_messages

Атрибут	Тип данных	Необязательное поле	Примечание
id	uuid	No	Идентификатор сообщения
conversation_id	uuid	No	Ссылка на диалог
role_id	uuid	No	Ссылка на роль сообщения
content	text	No	Содержимое сообщения
created_at	timestamp	No	Дата создания сообщения